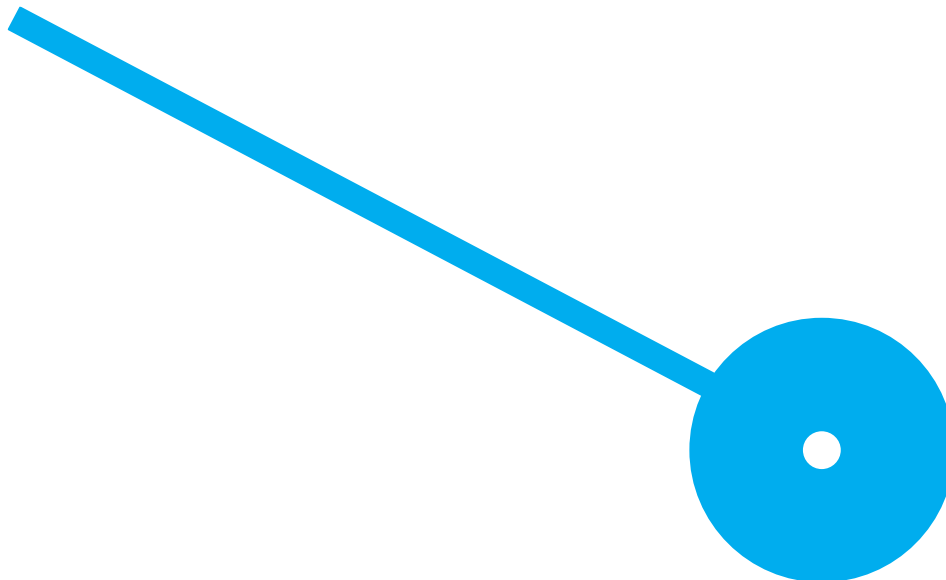




FastBite- PEDWM

Alexandre Freitas (8220154), Duarte Sousa
(8220160), Paulo Coelho (8220195)

05/2026



Resumo

O presente projeto consiste na concepção, desenvolvimento e implementação da FastBite, uma plataforma digital para gestão de encomendas e entregas de refeições, permitindo a interação coordenada e em tempo real entre três classes de utilizadores com perfis distintos: clientes, restaurantes e estafetas.

O principal desafio técnico do sistema reside na sincronização de estado num ambiente multiutilizador distribuído, onde uma mesma encomenda é acompanhada simultaneamente pelos três intervenientes e cada transição (confirmação do restaurante, atribuição do estafeta, recolha, trânsito, entrega) tem de ser validada centralmente, persistida de forma atômica e propagada com baixa latência. Para responder a este problema, adotou-se uma arquitetura orientada a eventos combinando uma API GraphQL para operações estruturadas com WebSockets para comunicação bidirecional em tempo real.

O *backend* foi desenvolvido em Laravel (PHP), com persistência em PostgreSQL, expondo o schema GraphQL através do package Lighthouse e implementando a camada de tempo real através de Workerman/GatewayWorker, organizada em canais segmentados por utilizador, restaurante, encomenda, entrega e chat.

O sistema oferece dois clientes complementares: um *frontend web* em React/Vite dedicado à gestão operacional do restaurante (fila de pedidos, cozinha virtual, campanhas, estatísticas) e um *frontend mobile* em React Native/Expo que cobre, num só binário, os fluxos de cliente (descoberta, carrinho, checkout, *tracking*) e de estafeta (disponibilidade, ofertas, fluxo de entrega e transmissão contínua de localização).

Os resultados demonstram a eficácia da combinação de tecnologias modernas e arquiteturas reativas na construção de aplicações em tempo real, evidenciando a relevância da modelação explícita do domínio, da separação de responsabilidades e da utilização de eventos no desenho de sistemas robustos e extensíveis.

Abstract

This project consists of the design, development, and implementation of FastBite, a digital platform for managing meal orders and deliveries, enabling coordinated real-time interaction between three distinct user groups: customers, restaurants, and couriers.

The main technical challenge of the system lies in state synchronization within a distributed multi-user environment, where the same order is monitored simultaneously by the three participants. Each transition, such as restaurant confirmation, courier assignment, pickup, transit, and delivery, must be centrally validated, atomically persisted, and propagated with low latency. To address this challenge, an event-driven architecture was adopted, combining a GraphQL API for structured operations with WebSockets for bidirectional real-time communication.

The backend was developed in Laravel (PHP), with persistence in PostgreSQL, exposing the GraphQL schema through the Lighthouse package and implementing the real-time layer through Workerman/GatewayWorker, organized into channels segmented by user, restaurant, order, delivery, and chat.

The system provides two complementary clients: a web frontend built with React/Vite, dedicated to the restaurant's operational management, including order queue, virtual kitchen, campaigns, and statistics; and a mobile frontend built with React Native/Expo, which covers, in a single binary, both customer flows, such as discovery, cart, checkout, and tracking, and courier flows, such as availability, offers, delivery workflow, and continuous location transmission.

The results demonstrate the effectiveness of combining modern technologies with reactive architectures in the development of real-time applications, highlighting the importance of explicit domain modelling, separation of responsibilities, and the use of events in the design of robust and extensible systems.

Índice

Resumo.....	2
Abstract.....	3
Lista de Figuras.....	6
Lista de Tabelas.....	7
Capítulo I.....	8
Contextualização e Motivação.....	8
1.1 Introdução	8
1.1.2 Objetivos.....	8
1.1.3 Resultados.....	9
1.1.3 Organização do Documento.....	9
1.2 Enquadramento Tecnológico.....	9
1.2.1 Laravel (Backend)	9
1.2.2 GraphQL.....	10
1.2.3 WebSockets.....	10
1.2.4 <i>React/Vite no Frontend Web</i>	10
1.2.5 React Native/Expo no Frontend Mobile	10
1.2.6 Paradigmas de Programação	11
1.2.7 Plataformas de Entrega.....	12
1.3 Conhecimento Prévio das Tecnologias.....	12
Capítulo 2.....	14
Conceptualização do Problema.....	14
2.1 A Plataforma FastBite.....	14
2.1.1 Atores do Sistema.....	14
2.1.2 Ciclo de Vida de uma Encomenda.....	15
2.1.3 Modelação Conceptual do Domínio.....	15
2.2 Arquitetura Conceptual do Sistema	16
2.3.1 Comunicação Cliente-Servidor	17
2.4 Principais Desafios Técnicos.....	17
Capítulo 3.....	19
Modelação do Sistema.....	19
3.1 Introdução.....	19
3.2 Diagrama de Classes	19
3.2.2 Aplicação de Design Patterns.....	21
3.2.2.1 Padrão <i>State</i>	21
3.2.2.3 Padrão <i>Factory</i>	22

3.2.2.2 Padrão <i>Observer</i>	22
3.3 Diagrama ER.....	22
3.4 Diagramas de Sequência.....	25
3.4.1 Fluxo de Criação de Encomenda.....	25
3.4.2 Fluxo de Entrega em Tempo Real.....	26
3.5 Máquinas de Estado.....	27
3.5.1 Máquina de Estados – Order.....	27
3.5.2 Máquina de Estados – Payment.....	28
3.5.3 Máquina de Estados – Delivery.....	28
Capítulo 4.....	30
Implementação da Solução	30
4.1 Arquitetura Implementada.....	30
4.2 Implementação do Backend.....	30
4.2.1 Programação Funcional.....	30
4.2.2 Programação Orientada a Aspectos	31
4.2.3 Programação Orientada a Eventos.....	32
4.3 API GraphQL.....	33
4.4 Comunicação em Tempo Real com Workerman/GatewayWorker.....	34
4.5 Implementação dos Frontends	35
4.6 Organização do Projeto e Gestão do Trabalho.....	38
Capítulo 5.....	39
Discussão dos Resultados e Conclusão.....	39
5.1 Apresentação e discussão de resultados.....	39
5.2 Trabalho Futuro.....	39
5.3 Conclusão	40
Anexos.....	41
Anexo A – Frontend Web.....	41
Anexo B – Frontend Mobile Cliente.....	42
Anexo C – Frontend Mobile Estafeta.....	43

Lista de Figuras

Figura 1 Arquitetura Conceptual do Sistema	16
Figura 2 Diagrama de Classes.....	20
Figura 3 – Diagrama de classes – Enumerações.....	21
Figura 4 Diagrama ER	24
Figura 5 Diagrama de sequências de checkout de encomenda.....	25
Figura 6 Diagrama de sequências de entrega.....	26
Figura 7 Diagrama de Estados: Order	27
Figura 8 Diagrama de Estados: Payment.....	28
Figura 9 Diagrama de Estados: Delivery	28
Figura 10 Frontend Dashboard Restaurante	36
Figura 11 Frontend Cozinha Virtual.....	36
Figura 12 Frontend Cliente Home e Carrinho.....	37
Figura 13 Frontend Estafeta A Recolher Encomenda e Aceitar Encomenda.....	38
Figura 14 Frontend Restaurante campanhas	41
Figura 15 Frontend Catálogo Produtos Restaurante.....	41
Figura 16 Frontend Cliente Carrinho e Avaliação.....	42
Figura 17 Frontend Estafeta A Chegar e Pedido Pronto a Recolher.....	43
Figura 18 Frontend Estafeta.....	44

Lista de Tabelas

Tabela 1 Atores do sistema.....	15
---------------------------------	----

Capítulo I

Contextualização e Motivação

1.1 Introdução

No contexto da unidade curricular de Paradigmas Emergentes para o Desenvolvimento Web e Mobile, foi proposto o desenvolvimento de um sistema que integrasse diferentes paradigmas de programação e tecnologias modernas de comunicação.

Neste âmbito, o presente projeto consiste na conceção e desenvolvimento da plataforma FastBite, um sistema de gestão de encomendas e entregas de refeições, permitindo a interação em tempo real entre clientes, restaurantes e estafetas.

Atualmente, plataformas de *delivery* enfrentam desafios significativos relacionados com:

- sincronização de estados entre múltiplos intervenientes;
- atualização em tempo real do estado das encomendas;
- escalabilidade do sistema face a múltiplos pedidos concorrentes;
- gestão eficiente de eventos e notificações.

A solução proposta baseia-se numa arquitetura moderna orientada a eventos, combinando GraphQL para operações estruturadas e WebSockets para comunicação em tempo real, permitindo garantir uma experiência fluida e responsiva para todos os utilizadores.

1.1.2 Objetivos

O principal objetivo deste projeto consiste na conceção e implementação de uma aplicação que permita a gestão completa do ciclo de vida de encomendas de refeições, explorando paradigmas modernos de desenvolvimento e princípios de engenharia de software.

De forma mais específica, os objetivos incluem:

- Conceber uma arquitetura de software que aplique os paradigmas de Programação Funcional, Programação Orientada a Aspectos e Programação Event-Driven;
- Implementar um *backend* em Laravel responsável pela lógica de negócio, gestão de utilizadores e coordenação entre restaurantes e estafetas;
- Desenvolver um *frontend web* em React e um *frontend mobile* em React Native capazes de interagir com o *backend* de forma reativa;
- Utilizar GraphQL para consultas e mutações às entidades do sistema;
- Implementar comunicação em tempo real através de WebSockets para a atualização instantânea do estado das encomendas;
- Modelar o domínio do sistema de entregas de forma estruturada, incluindo entidades como clientes, pedidos e entregas;
- Documentar a arquitetura e a modelação da solução através de diagramas UML e descrição técnica detalhada.

1.1.3 Resultados

Como resultado do desenvolvimento do projeto, foram obtidos os seguintes artefactos e componentes técnicos:

- Implementação de uma plataforma funcional de gestão de encomendas (FastBite);
- Implementação de um *backend* em Laravel, responsável pela lógica do sistema e suporte a GraphQL e WebSockets;
- Implementação de interfaces de utilizador através de um *frontend web* (React) e um *frontend mobile* (React Native) totalmente integrados;
- Definição de uma API GraphQL para consulta e execução de operações sobre as entidades do sistema;
- Implementação de comunicação em tempo real através de WebSockets;
- Modelação do domínio da aplicação através de entidades estruturadas representativas do ciclo de entrega;
- Aplicação prática de Programação Funcional, Programação Orientada a Aspectos e Programação Event-Driven;
- Documentação técnica detalhada do sistema na forma de um relatório completo.

1.1.3 Organização do Documento

O presente relatório encontra-se organizado da seguinte forma:

- **Capítulo 1 – Contextualização e Motivação**
Apresenta o enquadramento do projeto, os objetivos definidos e a organização geral do documento.
- **Capítulo 2 – Conceptualização do Problema**
Descreve o domínio da aplicação, os principais requisitos do sistema e a arquitetura conceptual adotada.
- **Capítulo 3 – Modelação do Sistema**
Apresenta a modelação técnica da aplicação, incluindo diagramas UML como diagramas de classes, sequência e estados.
- **Capítulo 4 – Implementação da Solução**
Descreve as tecnologias utilizadas, a estrutura do sistema e as principais decisões técnicas tomadas durante o desenvolvimento.
- **Capítulo 5 – Discussão dos Resultados**
Apresenta uma análise dos resultados obtidos, bem como os principais desafios técnicos enfrentados durante o desenvolvimento, com a sua reflexão crítica.

1.2 Enquadramento Tecnológico

1.2.1 Laravel (Backend)

O Laravel (PHP) foi adotado como *framework* base do *backend* pela maturidade do seu ecossistema e pela amplitude de adoção na indústria, o que assegura disponibilidade de documentação, *packages* e respostas a problemas comuns. A arquitetura do *backend* tira partido de vários componentes da *framework*:

- Eloquent ORM: representação orientada a objetos das tabelas com relações expressas declarativamente;
- Migrations: controlo de versão da estrutura da base de dados;
- Jobs e Queues: processamento assíncrono de tarefas críticas;
- Events e Listeners: integração nativa com o sistema de eventos do Laravel, utilizado como camada de orquestração entre o domínio e a infraestrutura de tempo real;

- Docker/Sail: ambiente de desenvolvimento containerizado, garantindo paridade entre máquinas e com a configuração de teste.

1.2.2 GraphQL

GraphQL é uma linguagem de consulta para APIs e um *runtime* desenvolvido originalmente pelo Facebook, concebido para permitir que os clientes solicitem exatamente os dados de que necessitam.

Ao contrário das APIs REST tradicionais, onde cada *endpoint* devolve uma estrutura de dados fixa, o GraphQL permite que o cliente especifique explicitamente quais os campos que pretende obter.

Esta abordagem reduz o problema de *over-fetching* e *under-fetching*, permitindo uma comunicação mais eficiente entre cliente e servidor.

Outra característica importante do GraphQL é a distinção entre três tipos principais de operações:

- *Queries*, utilizadas para obter dados;
- *Mutations*, utilizadas para alterar o estado do sistema;
- *Subscriptions*, utilizadas para receber atualizações em tempo real.

1.2.3 WebSockets

WebSockets constituem um protocolo de comunicação que permite estabelecer uma ligação bidirecional persistente entre cliente e servidor. Ao contrário do modelo tradicional baseado em HTTP, onde cada interação requer um novo estabelecimento de comunicação, os WebSockets permitem a troca contínua de mensagens entre ambas as partes após o estabelecimento da conexão.

Esta característica torna os WebSockets particularmente adequados para aplicações que exigem comunicação em tempo real, tais como sistemas de *chat*, jogos *online* ou *dashboards* interativos.

1.2.4 React/Vite no Frontend Web

O *frontend* web foi desenvolvido com React em conjunto com o *bundler* Vite, focando-se na interface de gestão do estabelecimento. Esta plataforma centraliza as operações do restaurante:

- Gestão de Encomendas: pedidos ativos, histórico e detalhe dos mesmos;
- Cozinha Virtual: visualização orientada ao fluxo de preparação dos itens individuais;
- Gestão de Menu: administração de catálogo, categorias, opções e disponibilidade;
- Marketing e Qualidade: criação de campanhas, gestão de cupões e visualização de avaliações;
- Comunicação: chat com cliente e centro de notificações.

1.2.5 React Native/Expo no Frontend Mobile

A aplicação mobile foi desenvolvida em React Native com recurso ao ecossistema Expo, permitindo a criação de aplicações nativas para iOS e Android a partir de uma única base de código Javascript. A mesma aplicação Expo cobre os dois perfis numa só aplicação:

- Fluxo do Cliente: *home*, listagem de restaurantes, menus, gestão de carrinho, *checkout*, *tracking* da entrega em tempo real e histórico;
- Fluxo do Estafeta: gestão de disponibilidade, receção de ofertas de entrega, confirmação de recolha, fluxo de entrega e transmissão contínua de localização.

1.2.6 Paradigmas de Programação

O desenvolvimento do sistema foi guiado por diferentes paradigmas de programação, escolhidos para dar resposta aos desafios específicos de cada componente da aplicação. A utilização combinada destes paradigmas permite a criação de um sistema escalável, reativo e de fácil manutenção.

Programação Funcional

A Programação Funcional é um paradigma de programação que privilegia a construção de software através da composição de funções, dando preferência a funções puras, imutabilidade dos dados e ausência de efeitos secundários. Este paradigma facilita a previsibilidade do comportamento do sistema, uma vez que uma função pura devolve sempre o mesmo resultado para os mesmos argumentos e não depende de estado externo.

- **Aplicação no Projeto:** Utilizada de forma complementar em ambas as camadas do sistema. No backend, classes como PricingCalculator e GeoMath isolam regras de cálculo em funções puras e determinísticas, métodos como calculateSubtotal, calculateTotal e distanceKm não dependem de estado externo e devolvem sempre o mesmo resultado para os mesmos argumentos. São também usadas funções de ordem superior (pipe, sumBy), closures e transformações declarativas sobre Collections do Laravel (map, filter, flatMap, sum).

Programação Orientada a Aspetos (AOP)

A Programação Orientada a Aspetos é um paradigma que tem como objetivo separar da lógica principal da aplicação as preocupações transversais, também conhecidas como *cross-cutting concerns*. Estas preocupações incluem funcionalidades como registo de *logs*, auditoria, segurança, validação, controlo de permissões e gestão de transações, que tendem a aparecer repetidamente em várias partes do sistema. Ao encapsular estas funcionalidades em aspetos independentes, este paradigma permite manter a lógica de negócio mais limpa, modular e fácil de manter, evitando a repetição de código em múltiplos métodos ou serviços.

- **Aplicação no Projeto:** No projeto, a Programação Orientada a Aspetos é aplicada no *backend* através da biblioteca ray/aop, sobretudo para implementar mecanismos de *logging* e para tornar determinadas funções transacionais. Esta abordagem permite associar comportamentos transversais a métodos ou serviços específicos sem alterar diretamente a sua lógica principal.

Programação Orientada a Eventos

A Programação Orientada a Eventos (POE) é um paradigma de programação em que o fluxo de execução da aplicação é determinado principalmente pela ocorrência de eventos, que podem resultar de ações do utilizador ou de processos internos do sistema. Em vez de seguir apenas uma sequência fixa de instruções, a aplicação define previamente comportamentos, conhecidos como *event handlers*, que são executados em resposta ao disparo desses eventos.

- **Aplicação no Projeto:** A Programação Orientada a Eventos foi aplicada no *backend* para reagir automaticamente a alterações importantes do sistema. Um dos principais exemplos ocorre quando o estado de uma encomenda é alterado. No modelo Order, a função transitionTo() atualiza o estado da encomenda e, após a confirmação da transação na base de dados, executa o *handler* OrderStatusChangedHandler. Este *handler* regista o acontecimento no histórico da encomenda e cria um evento no sistema de *outbox*.

1.2.7 Plataformas de Entrega

Atualmente, as plataformas digitais de entrega de refeições (tais como Uber Eats, Glovo ou Bolt Food) assumem um papel central no ecossistema da restauração e na economia de partilha. Estas plataformas caracterizam-se pela sua elevada complexidade logística e tecnológica, uma vez que necessitam de orquestrar três intervenientes distintos em simultâneo: o cliente final, o restaurante e o estafeta. Para garantir uma experiência fluida, dependem de arquiteturas distribuídas e de tecnologias de comunicação em tempo real, essenciais para a sincronização de dados e rastreamento (*tracking*) de localização.

Frequentemente, estas plataformas recorrem a arquiteturas baseadas em APIs REST convencionais em conjunto com serviços de terceiros ou mecanismos de *polling* para a atualização de coordenadas GPS e estados dos pedidos. No entanto, o uso extensivo de *polling* em redes móveis pode introduzir latência indesejada e resultar num consumo excessivo de bateria e de dados nos dispositivos dos utilizadores.

No presente projeto, foi adotada uma abordagem híbrida e otimizada, baseada na utilização de GraphQL para as operações de consulta (como listagem de restaurantes e ementas) e execução de comandos (como a submissão de um carrinho), combinada com WebSockets para a propagação de eventos críticos e localização em tempo real. Esta arquitetura não só resolve o problema de *over-fetching*, algo fundamental para garantir a performance na aplicação móvel, como permite separar claramente as operações estruturadas do fluxo de eventos assíncronos e de alta frequência.

A conjugação de diferentes paradigmas de programação, como a Orientação a Eventos para reagir ao ciclo de vida da encomenda e a Orientação a Aspectos, constitui também um fator diferenciador da abordagem adotada, permitindo desacoplar a lógica de negócio das preocupações infraestruturais e facilitando a evolução futura e a escalabilidade do sistema.

1.3 Conhecimento Prévio das Tecnologias

Relativamente às tecnologias e *frameworks* adotadas no desenvolvimento do projeto, os elementos do grupo não possuíam experiência prévia com as principais ferramentas escolhidas. Este cenário implicou um processo de aprendizagem contínuo, autónomo e desafiante ao longo de todo o trabalho.

A *framework* Laravel, utilizada na construção do *backend*, constituiu uma novidade absoluta para a equipa. A sua adoção exigiu não só a familiarização com as particularidades do ecossistema PHP moderno, mas também a assimilação de ferramentas e conceitos arquiteturais inerentes à *framework*. Foi necessário investir tempo na compreensão do ORM Eloquent para interação com a base de dados bem como na configuração e orquestração do ambiente de desenvolvimento com recurso ao Docker e Laravel Sail.

No que diz respeito ao desenvolvimento do *frontend*, tanto o React (utilizado na plataforma web) como o React Native (utilizado na aplicação móvel) representavam tecnologias nunca exploradas pelos membros do grupo. O domínio destas ferramentas obrigou a uma aprendizagem aprofundada dos conceitos fundamentais da biblioteca, nomeadamente a sua abordagem declarativa baseada em componentes, a gestão de estado imutável e a utilização de *Hooks*.

Adicionalmente, a vertente *mobile* introduziu desafios específicos, exigindo a familiarização com o ecossistema Expo e com as particularidades do desenvolvimento desta aplicação para dispositivos móveis tais como a integração de mapas e a captação de localização.

De um modo geral, a ausência de contacto prévio com estas *frameworks* transformou o desenvolvimento deste projeto numa oportunidade para a consolidação e expansão das competências técnicas do grupo. Este contexto promoveu não só a aprendizagem de novas tecnologias com elevada relevância na indústria de desenvolvimento de *software*, mas também a sua aplicação prática e integrada na construção de um sistema de entregas.

Capítulo 2

Conceptualização do Problema

O desenvolvimento de plataformas de encomendas e entregas de refeições apresenta desafios técnicos significativos, especialmente quando múltiplos atores com papéis distintos necessitam de interagir simultaneamente sobre um estado partilhado do sistema. Sistemas de *delivery* representam um exemplo claro deste tipo de aplicações, uma vez que exigem coordenação contínua entre clientes, restaurantes e estafetas, validação centralizada de regras de negócio e atualização imediata da informação apresentada a cada participante.

2.1 A Plataforma FastBite

No caso específico da FastBite, a implementação envolve a gestão de vários elementos do domínio da plataforma, incluindo utilizadores com diferentes perfis, restaurantes e respetivos catálogos, carrinhos de compras, encomendas, pagamentos, entregas, promoções, notificações, chat e avaliações. Além disso, é necessário garantir que as regras de negócio são aplicadas corretamente pelo sistema, por exemplo, que uma encomenda não pode ser entregue sem antes ter sido preparada, e que todos os participantes recebem atualizações do estado de forma consistente e em tempo real.

Outro desafio relevante consiste na gestão simultânea de múltiplos fluxos independentes. Num dado momento, vários clientes podem estar a criar encomendas, vários restaurantes podem estar a preparar pedidos e vários estafetas podem estar em trânsito, sendo necessário isolar o estado de cada processo e evitar interferência entre fluxos diferentes.

Tendo em conta estes requisitos, foi necessário conceber uma arquitetura que permitisse:

- manter um estado consistente de encomendas, pagamentos e entregas no servidor;
- suportar comunicação em tempo real entre clientes, restaurantes e estafetas;
- gerir múltiplos fluxos concorrentes de forma isolada;
- aplicar corretamente as regras de negócio e transições de estado;
- separar claramente a lógica de domínio da infraestrutura de comunicação.

2.1.1 Atores do Sistema

O sistema é composto por três atores principais: Cliente, Restaurante e Estafeta. Cada um destes atores interage com a plataforma através de interfaces específicas e desempenha um conjunto distinto de responsabilidades no fluxo de funcionamento da aplicação.

Ator	Plataforma	Responsabilidades principais
Cliente	Mobile (React Native/Expo)	Procurar restaurantes, explorar catálogos e menus, gerir carrinho com opções, finalizar encomenda, efetuar pagamento, acompanhar encomenda em tempo real, comunicar via chat, avaliar serviço.
Restaurante	Web (React/Vite)	Receber encomendas em tempo real, aceitar/rejeitar pedidos, atualizar estado de preparação, gerir catálogo de produtos, opções e categorias; gerir campanhas e cupões, consultar avaliações, consultar estatísticas e faturação.

Estafeta	Mobile (React Native/Expo)	Indicar disponibilidade, receber ofertas de entrega, aceitar/rejeitar, confirmar recolha no restaurante; transmitir localização em tempo real, marcar entrega como concluída, comunicar com cliente se necessário.
----------	----------------------------	--

Tabela 1 Atores do sistema

2.1.2 Ciclo de Vida de uma Encomenda

O fluxo central da FastBite é o ciclo de vida de uma encomenda, que envolve a participação coordenada dos três atores principais do sistema.

O processo inicia-se quando um cliente consulta os restaurantes disponíveis, explora os respetivos catálogos de produtos e adiciona itens ao seu carrinho de compras. Cada item pode incluir opções configuráveis, como ingredientes adicionais ou variações, que influenciam o preço final. Após compor o carrinho, o cliente procede ao checkout, momento em que o sistema calcula o total da encomenda, aplica eventuais promoções ou cupões de desconto e cria a encomenda com estado inicial pendente.

Segue-se a fase de pagamento, na qual o sistema regista o método escolhido pelo cliente e simula o processamento da transação. O pagamento possui o seu próprio ciclo de estados, pendente, concluído ou falhado, e pode expirar caso não seja finalizado dentro de um determinado período. Após a confirmação do pagamento, o sistema inicia o processo de atribuição de um estafeta antes de notificar o restaurante. Para isso, é criada uma oferta de entrega associada à encomenda, enviada a estafetas disponíveis na proximidade do restaurante. Cada estafeta pode aceitar ou rejeitar a oferta, caso a oferta expire sem aceitação, o sistema pode gerar novas ofertas para outros estafetas disponíveis.

Quando um estafeta aceita a oferta, é criada a entrega associada à encomenda. Só nesse momento é que o restaurante é notificado da nova encomenda, podendo então aceitar ou rejeitar o pedido. Caso aceite, a encomenda transita para o estado de preparação, durante o qual o restaurante atualiza o progresso dos itens individuais. Quando todos os itens estão prontos, a encomenda é marcada como pronta para recolha.

O estafeta desloca-se então ao restaurante, recolhe o pedido e inicia o transporte até ao cliente. Durante todo este percurso, o estafeta envia atualizações de localização que alimentam o sistema de *tracking*, permitindo ao cliente acompanhar a entrega em tempo real num mapa.

Após a entrega, o cliente pode avaliar tanto o restaurante como o estafeta, contribuindo para o sistema de reputação da plataforma.

2.1.3 Modelação Conceptual do Domínio

Para garantir uma representação clara e organizada do sistema, foi realizada uma modelação do domínio baseada nos principais conceitos da plataforma de entregas. Esta modelação permite identificar as entidades fundamentais do sistema e as relações existentes entre elas.

Os principais elementos do domínio identificados são:

- **User** — utilizador do sistema, podendo atuar como cliente ou assumir perfis específicos;
- **Courier** — perfil de estafeta responsável pela aceitação e realização de entregas;
- **ChainManager** — perfil de gestor de uma cadeia de restaurantes;

- **LocalManager** – perfil de gestor de um restaurante individual;
- **UserAddress** – moradas de entrega do utilizador
- **RestaurantChain** – cadeia de restaurantes que partilha catálogo, categorias, produtos e campanhas;
- **Restaurant + RestaurantAddress** – estabelecimento individual pertencente a uma cadeia, incluindo a sua morada física;
- **Category** – categoria de produtos pertencente ao catálogo de uma cadeia;
- **Product + ProductOptionGroup + ProductOption** – produto do catálogo e respetivas opções configuráveis, como tamanhos, extras ou acompanhamentos;
- **RestaurantProduct** – associação entre Product e Restaurant, permitindo definir preço local, disponibilidade e tempo estimado de preparação;
- **Cart + CartItem + CartItemOption** – carrinho ativo do cliente, com os itens adicionados e respetivas opções configuradas;
- **Order + OrderItem + OrderItemOption** – encomenda finalizada, incluindo os seus itens e opções escolhidas;
- **OrderAddress** – snapshot da morada de entrega usada numa encomenda;
- **OrderDiscount** – desconto efetivamente aplicado a uma encomenda ou item da encomenda;
- **OrderEvent** – registo de eventos associados ao ciclo de vida da encomenda;
- **Payment + PaymentEvent** – pagamento associado a uma encomenda e respetivo histórico de eventos;
- **Delivery + DeliveryEvent** – entrega física associada a uma encomenda e respetivo histórico de eventos;
- **DeliveryOffer** – oferta temporizada de entrega enviada a um estafeta;
- **CourierPositionHistory** – histórico das posições do estafeta durante uma entrega;
- **Promotion + Coupon + PromotionItem** – campanhas promocionais, cupões de desconto e associação destes a produtos ou categorias abrangidas;
- **Notification** – notificação enviada a um utilizador, tanto in-app como por push;
- **Chat + ChatParticipant + Message** – conversa contextual associada a uma encomenda, incluindo participantes e mensagens;
- **Review** – avaliação feita por um cliente a um restaurante ou estafeta;
- **OutboxEvent** – evento interno usado para comunicação assíncrona, publicação de eventos e integração com mecanismos de notificação e tempo real.

2.2 Arquitetura Conceptual do Sistema

A arquitetura segue um modelo cliente-servidor com múltiplos clientes heterogéneos, com separação clara entre interfaces de utilizador e lógica de negócio.

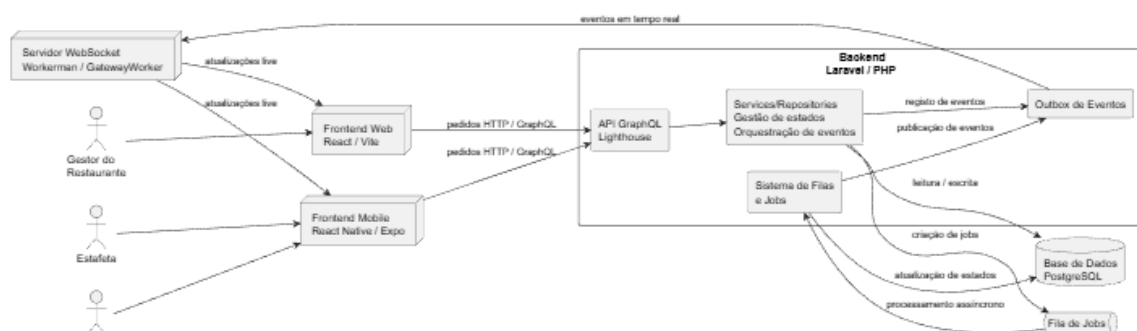


Figura 1 Arquitetura Conceptual do Sistema

De forma conceptual, a arquitetura é composta pelos seguintes elementos:

- *Frontend Web* (React/Vite) — interface de gestão do restaurante;
- *Frontend Mobile* (React Native/Expo) — aplicação única para Cliente e Estafeta, com fluxos distintos;
- *Backend* (Laravel / PHP) — lógica de negócio, gestão de estados, orquestração de eventos;
- API GraphQL (Lighthouse) — operações estruturadas e tipadas;
- Servidor WebSocket (Workerman/GatewayWorker) — propagação de eventos em tempo real;
- Base de dados (PostgreSQL) — persistência relacional;
- Sistema de Filas e Jobs — processamento assíncrono (expiração de pagamentos e ofertas, atribuição de estafetas e publicação de eventos *outbox*).

2.3.1 Comunicação Cliente-Servidor

A comunicação entre clientes e servidor é realizada através de duas tecnologias complementares. A primeira corresponde à utilização de GraphQL, que permite executar operações e consultas estruturadas sobre o sistema, como a criação de encomendas, consulta de restaurantes, adição de itens ao carrinho, checkout e consulta de histórico. O GraphQL permite que cada interface solicite exatamente os dados de que necessita para cada ecrã, reduzindo o *over-fetching* evitando a proliferação de *endpoints* demasiado específicos.

A segunda corresponde à utilização de WebSockets através de GatewayWorker/Workerman, que permitem a distribuição de eventos em tempo real para todos os utilizadores ligados. Desta forma, quando ocorre uma ação relevante no sistema, por exemplo, quando um restaurante confirma uma encomenda, quando um estafeta aceita uma oferta de entrega ou quando a localização do estafeta é atualizada, o servidor publica um evento para os canais correspondentes, garantindo que todos os participantes visualizam informação atualizada sem necessidade de *polling*.

Os canais WebSocket estão organizados por contexto: canais por utilizador para notificações pessoais, canais por restaurante para novas encomendas, canais por encomenda para atualizações de estado, canais por entrega para *tracking* e canais por chat para mensagens em tempo real.

Esta abordagem reduz significativamente a latência na atualização do estado e melhora a experiência de todos os atores do sistema.

2.4 Principais Desafios Técnicos

Durante a conceção do sistema foram identificados vários desafios técnicos associados à implementação de uma plataforma de entregas com múltiplos atores e comunicação em tempo real.

Um dos principais desafios consiste na sincronização de estado entre múltiplos participantes. Uma mesma encomenda é acompanhada simultaneamente pelo cliente, pelo restaurante e pelo estafeta, sendo necessário garantir que todas as ações executadas são processadas pelo servidor de forma consistente e que as atualizações são propagadas corretamente para todos os intervenientes.

A consistência das transições de estado constitui outro desafio técnico importante. O sistema deve impedir que ações inválidas corrompam o estado do domínio, como cancelar uma encomenda já entregue ou aceitar uma oferta de entrega já expirada. Para tal, é necessária uma modelação adequada através de máquinas de estados com validação rigorosa em cada transição.

A fiabilidade na publicação de eventos representa também um desafio relevante. Quando o estado de uma encomenda muda, é fundamental garantir que o evento correspondente é efetivamente publicado para os canais WebSocket e para o sistema de notificações. O padrão Outbox foi adotado para garantir que eventos persistidos são publicados de forma controlada, mesmo em cenários de falha parcial.

Capítulo 3

Modelação do Sistema

3.1 Introdução

Neste capítulo é apresentada a modelação detalhada do sistema desenvolvido, com o objetivo de descrever a arquitetura e o comportamento da aplicação de forma estruturada.

Para tal, foram utilizados diversos tipos de diagramas UML, nomeadamente:

- diagramas de classes, para representar a estrutura estática do sistema;
- diagrama ER, para representar a estrutura de persistência da aplicação;
- diagramas de sequência, para modelar os fluxos de interação;
- diagramas de estados, para descrever o ciclo de vida das entidades principais.

3.2 Diagrama de Classes

Na Figura 2 Diagrama de Classes é possível visualizar o diagrama de classes que representa a estrutura do sistema ao nível do domínio, evidenciando as principais entidades, atributos e relações.

O modelo organiza as principais entidades do sistema, começando pelo User, que pode assumir diferentes perfis, como cliente, estafeta (Courier), gestor de cadeia (ChainManager) ou gestor local (LocalManager). O utilizador pode ter moradas, *tokens* de notificação, carrinhos, encomendas, avaliações, notificações e participações em chats.

A componente de restaurantes é estruturada a partir de RestaurantChain, que agrupa restaurantes, categorias, produtos, promoções e cupões. Cada restaurante possui uma morada, produtos disponíveis e gestores locais. O catálogo é composto por categorias, produtos, grupos de opções e opções adicionais, permitindo representar menus personalizáveis e produtos com preços ou disponibilidade específicos por restaurante.

O fluxo de compra é modelado através do Cart, dos respetivos itens e opções selecionadas. Quando o pedido é finalizado, é criada uma Order, que contém os produtos encomendados, a morada de entrega, descontos aplicados, eventos de estado, pagamento e eventual entrega. O pagamento é representado por Payment e pelos seus eventos, enquanto a entrega é representada por Delivery, podendo incluir ofertas para estafetas, histórico de localização e eventos da entrega.

O diagrama inclui ainda funcionalidades de suporte, como promoções, cupões, avaliações, notificações, chat entre atores e eventos assíncronos através de OutboxEvent. No geral, o modelo descreve de forma integrada todo o ciclo da aplicação, desde a gestão de utilizadores e restaurantes até à encomenda, pagamento, entrega, comunicação e notificações.

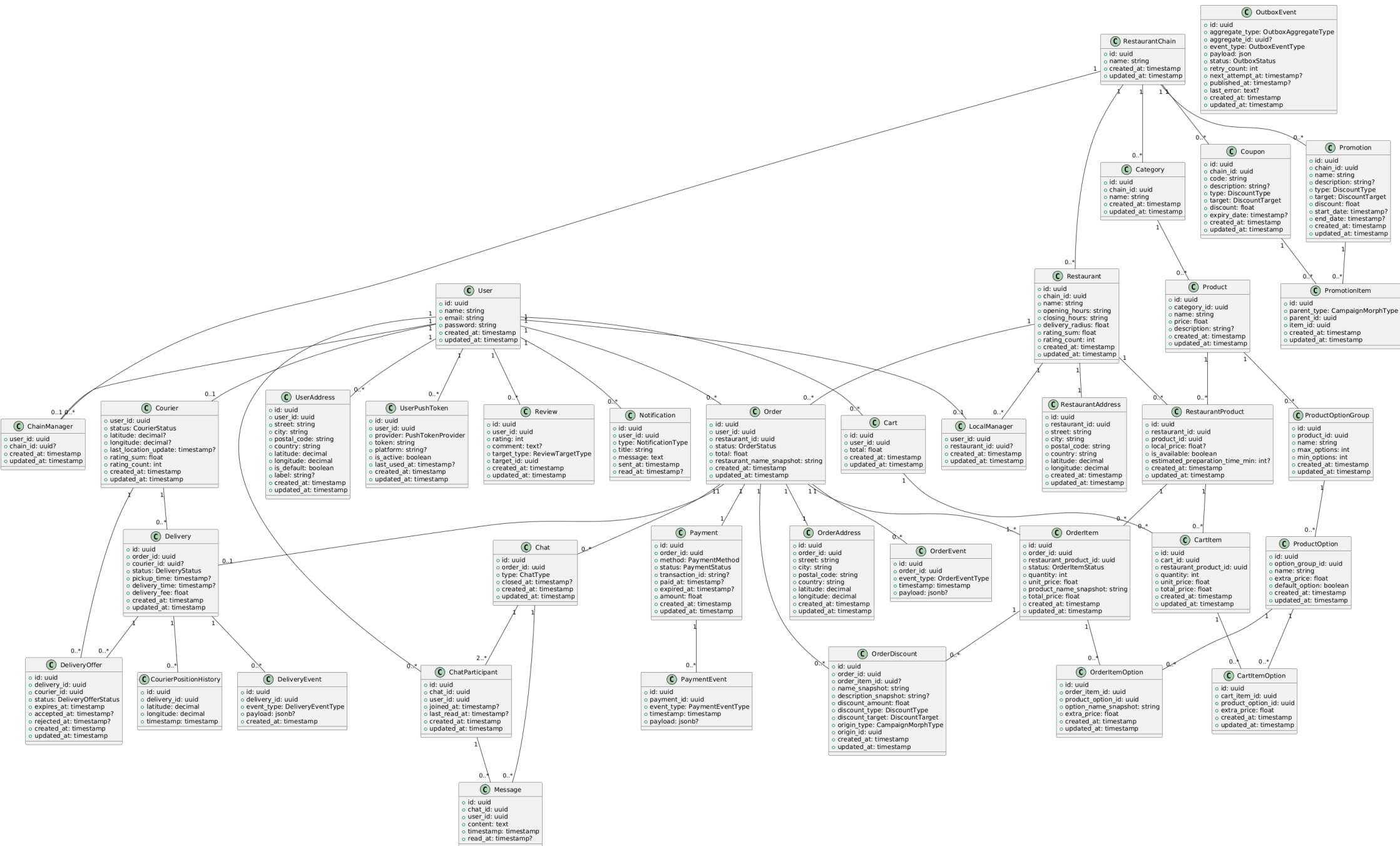


Figura 2 Diagrama de Classes

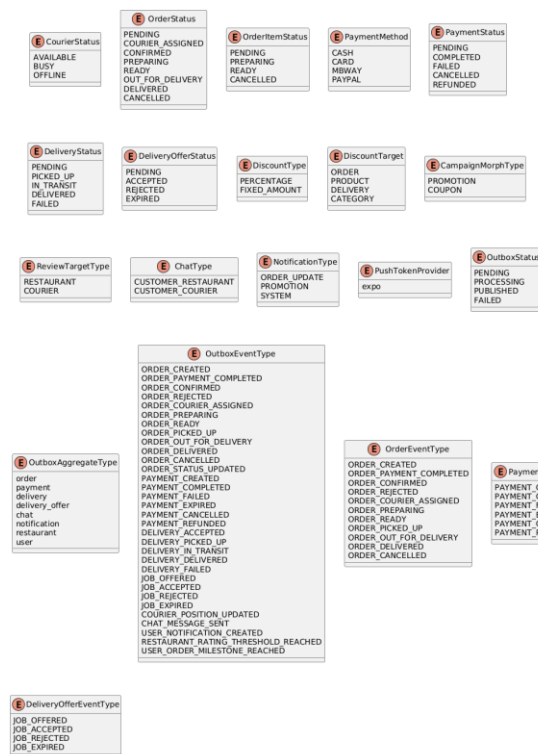


Figura 3 - Diagrama de classes - Enumerações

3.2.2 Aplicação de Design Patterns

Foram aplicados três design patterns clássicos para melhorar a organização, manutenibilidade e extensibilidade do sistema. Para cada padrão é apresentada a finalidade, a aplicação concreta na FastBite com referência às classes reais, e os benefícios obtidos.

3.2.2.1 Padrão State

O padrão *State* permite que um objeto altere o seu comportamento de acordo com o estado em que se encontra. O próprio estado passa a ser um objeto com comportamentos próprios, eliminando estruturas condicionais extensas (if/switch por estado) e centralizando as regras de transição.

O padrão foi aplicado de forma sistemática às três entidades cujo ciclo de vida é crítico: Order, Payment e Delivery, este padrão foi aplicado da seguinte forma:

- Interface base
- Classe abstrata — `AbstractOrderState`. Implementa a lógica comum de validação:
- Estados concretos — uma classe por estado (`PendingOrderState`, `ConfirmedOrderState`, `PreparingOrderState`, `ReadyOrderState`, `OutForDeliveryOrderState`, `DeliveredOrderState`, `CancelledOrderState`). Cada um declara apenas o seu `status()` e as suas `allowedTransitions()`:
- Factory de estados — `OrderStateFactory::from($status)`, que devolve a instância correta em função da enumeração atual.

Este padrão torna o sistema mais seguro e extensível, porque as regras de transição ficam concentradas nas classes de estado. Assim, adicionar ou alterar um estado não exige modificar toda a lógica dos serviços.

3.2.2.3 Padrão *Factory*

O padrão *Factory* é um padrão de criação que encapsula e centraliza a lógica de instanciação de objetos, permitindo criar diferentes classes concretas de forma flexível consoante parâmetros, configurações ou regras definidas pela aplicação. A aplicação mais evidente está nas máquinas de estado. As classes *OrderStateFactory*, *PaymentStateFactory* e *DeliveryStateFactory* recebem uma enumeração de estado e devolvem a instância correta do respetivo *State*.

Por exemplo, *OrderStateFactory::from(\$status)* transforma um *OrderStatus* como *PENDING*, *PREPARING* ou *DELIVERED* no objeto correspondente, como *PendingOrderState*, *PreparingOrderState* ou *DeliveredOrderState*. Isto evita que o resto do sistema tenha de conhecer todos os estados concretos ou repetir *match/switch* em vários serviços.

Este padrão melhora a organização do código, reduz duplicação e garante que a criação de objetos segue sempre uma regra centralizada.

3.2.2.2 Padrão *Observer*

O padrão *Observer* está presente na arquitetura orientada a eventos do backend. Sempre que ocorre uma alteração relevante no domínio, o sistema notifica automaticamente os componentes interessados sem que a lógica principal fique acoplada a eles.

Um exemplo direto está em *Order::transitionTo()*, onde, após a alteração do estado da encomenda, são executados handlers registados através de *Order::onStatusChanged()*. O handler principal, *OrderStatusChangedHandler*, cria o evento da encomenda, regista-o no histórico e envia-o para o *OutboxService*.

A nível global, o Laravel Event System também implementa este padrão. Em *AppServiceProvider*, eventos como *OutboxEventPublished*, *ChatMessageSent*, *CourierPositionUpdated*, *OrderStatusUpdated* e *UserNotificationCreated* são associados a *listeners* como *DispatchLaravelEventFromOutbox*, *DispatchSocketMessage* e *DispatchNotificationChannelsFromUserNotificationCreated*. Esta estrutura permite que notificações, WebSockets e outros efeitos secundários reajam aos eventos sem que os serviços de negócio conheçam diretamente esses mecanismos.

O principal benefício é o desacoplamento, o domínio apenas gera eventos ou escreve no *outbox*, enquanto os *listeners* tratam da publicação em tempo real, notificações e outras integrações.

3.3 Diagrama ER

O diagrama ER apresentado na Figura 4 Diagrama ER representa a estrutura de dados do sistema, evidenciando as principais entidades persistentes e as relações entre si no contexto da aplicação.

A tabela *users* funciona como ponto de referência para vários dados associados ao utilizador, como moradas, carrinhos, encomendas, avaliações, notificações, *tokens* de notificação e mensagens. Os perfis específicos, como estafetas e gestores, são representados em tabelas próprias ligadas a *users*, permitindo separar os dados comuns dos dados específicos de cada função. Da mesma forma, as cadeias de restaurantes, restaurantes, categorias e produtos são colocadas em tabelas distintas, evitando repetição de informação e facilitando a gestão do catálogo.

O modelo também distingue bem os dados temporários dos dados históricos. O carrinho guarda a seleção atual do utilizador, enquanto as encomendas registam uma versão consolidada da compra, incluindo morada, produtos, preços, descontos e estados. A existência de campos como `restaurant_name_snapshot`, `product_name_snapshot` e `option_name_snapshot` permite manter o histórico correto mesmo que os dados originais sejam alterados posteriormente.

As relações associadas a pagamentos, entregas, eventos, chats e notificações completam a estrutura da base de dados, permitindo acompanhar o ciclo completo de uma encomenda. Além disso, tabelas como `order_events`, `payment_events`, `delivery_events` e `outbox_events` ajudam a registar alterações importantes e suportam o processamento assíncrono de eventos.

Assim, o diagrama ER não descreve apenas as entidades do sistema, mas principalmente a forma como os dados são persistidos, relacionados e preservados ao longo do funcionamento da aplicação.

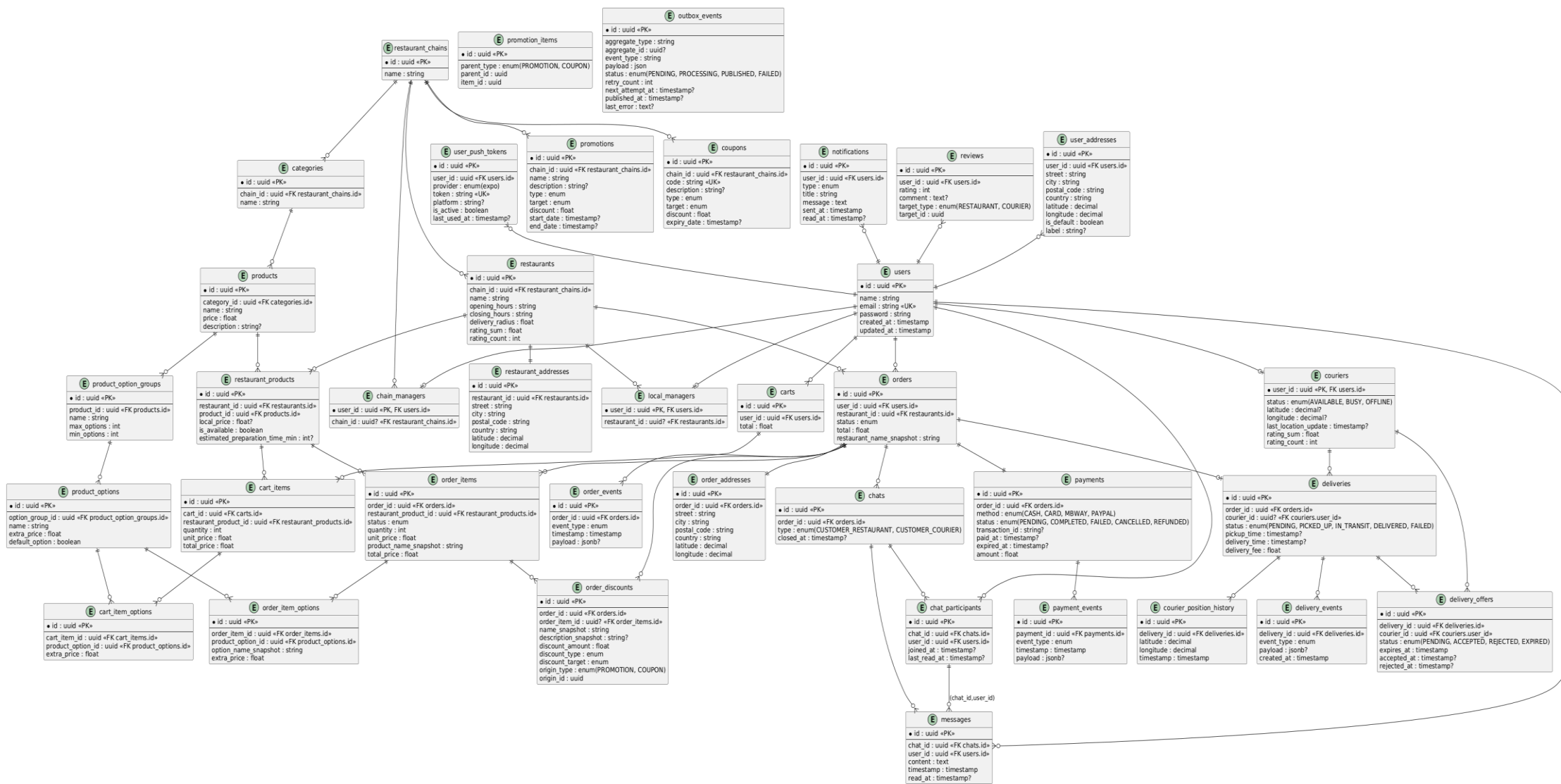


Figura 4 Diagrama ER

Os diagramas de sequência permitem representar de forma estruturada os principais fluxos de interação entre os componentes do sistema ao longo do tempo. São apresentados os dois fluxos mais representativos do FastBite.

```

sequenceDiagram
    participant Cliente
    participant Mobile as Mobile CustomerAppScreen
    participant GraphQL as GraphQL OrderQueries/OrderMutations
    participant OrderService
    participant OrderPricingService
    participant DeliveryService
    participant OutboxService
    participant Queue
    participant Repositories
    participant Gateway as Gateway/Worker via PublishOutboxEventJob
    participant Web as Web Restaurant

    Cliente->>Mobile: finaliza carrinho
    Mobile->>GraphQL: query previewCheckout(input)
    GraphQL->>OrderService: previewCheckout(userId, cartId, addressId, coupon)
    OrderService->>OrderPricingService: carrega carrinho, restaurante e morada  
calcula subtotal, entrega, descontos e total
    OrderPricingService->>GraphQL: CheckoutPreview
    GraphQL->>Mobile: totais e descontos
    Cliente->>Mobile: confirma checkout
    Mobile->>GraphQL: mutation checkoutOrder(input)
    GraphQL->>OrderService: checkoutOrder(userId, CheckoutDTO)
    OrderService->>OrderService: cria order PENDING, items e address snapshot
    OrderService->>Repositories: cria order, discounts quando aplicável
    OrderService->>Repositories: cria payment PENDING
    OrderService->>Repositories: order_events ORDER_CREATED
    OrderService->>Repositories: payment_events PAYMENT_CREATED
    OrderService->>Queue: enqueue ORDER_CREATED
    OrderService->>OutboxService: PublishOutboxEventJob(order_created)
    OrderService->>Queue: ExpirePendingPaymentJob(delay expired_at)
    OrderService->>OrderService: pagamento confirmado
    OrderService->>Repositories: order_events ORDER_PAYMENT_COMPLETED
    OrderService->>Queue: enqueue ORDER_PAYMENT_COMPLETED
    OrderService->>OutboxService: PublishOutboxEventJob(order_payment_completed)
    OrderService->>Repositories: payment_events PAYMENT_COMPLETED
    OrderService->>DeliveryService: createDeliveryForOrder(orderId, deliveryFee)
    DeliveryService->>Repositories: cria delivery PENDING
    OrderService->>Repositories: AssignCourierToDeliveryJob(deliveryId)
    OrderService->>Repositories: limpa carrinho
    OrderService->>GraphQL: order PENDING + payment
    GraphQL->>Mobile: order PENDING + payment
    OrderService->>OutboxService: PublishOutboxEventJob publica eventos de outbox
    OutboxService->>Gateway: ORDER_CREATED + ORDER_PAYMENT_COMPLETED
    Gateway->>Web: restaurant {restaurantId} orders
    Web->>DeliveryService: AssignCourierToDeliveryJob
    DeliveryService->>Repositories: procura estafeta disponível
    DeliveryService->>Repositories: cria delivery_offer PENDING
    DeliveryService->>Queue: enqueue JOB_OFFERED
    DeliveryService->>OutboxService: PublishOutboxEventJob(job_offered)
  
```

O diagrama de sequência do *checkout* representa o fluxo executado quando o cliente finaliza uma encomenda a partir do carrinho. Inicialmente, a aplicação mobile solicita uma pré-visualização do *checkout* através da *query* `previewCheckout`, permitindo calcular subtotal, taxa de entrega, descontos aplicáveis e total final antes da confirmação. Este cálculo é delegado ao `OrderPricingService`, enquanto o `OrderService` coordena o carregamento do carrinho, restaurante e morada de entrega.

Após a confirmação do pagamento, o sistema regista também os eventos `ORDER_PAYMENT_COMPLETED` e `PAYMENT_COMPLETED`. Nesta fase é criada a entrega associada à encomenda, inicialmente em estado `PENDING`, e é colocado na fila o job `AssignCourierToDeliveryJob`, responsável por iniciar o processo de atribuição de um estafeta. Os eventos relevantes são enviados para o `OutboxService`, que publica posteriormente as notificações através do `GatewayWorker`.

Por fim, o *frontend* recebe a resposta síncrona da *mutation* com a encomenda e o pagamento criados. Em paralelo, os eventos publicados pelo *outbox* são propagados em tempo real para os canais associados ao cliente, à encomenda e ao restaurante, permitindo que as interfaces sejam atualizadas automaticamente.

3.4.2 Fluxo de Entrega em Tempo Real

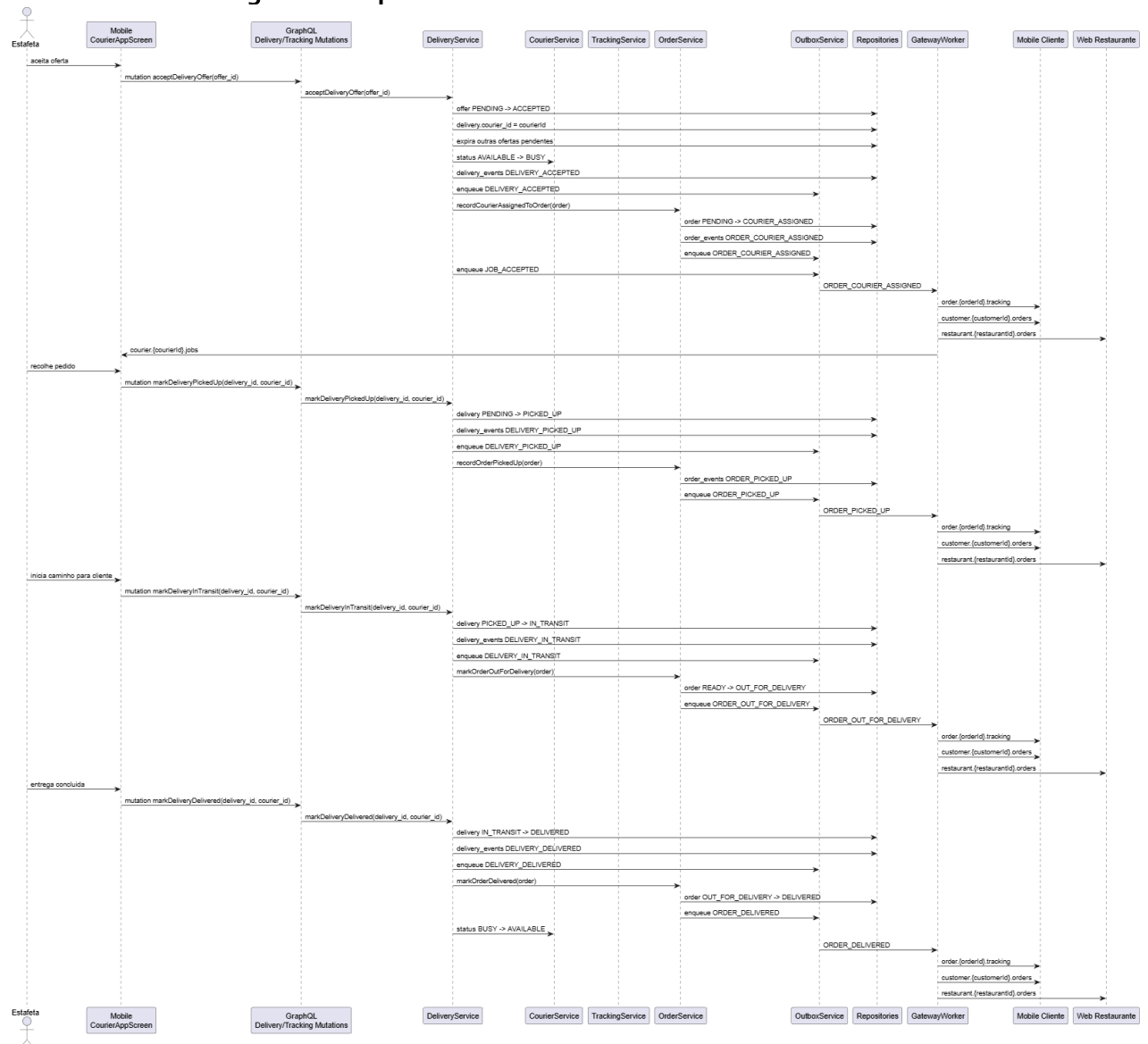


Figura 6 Diagrama de seqüências de entrega

O diagrama de seqüência do estafeta representa o ciclo de vida da entrega após a criação da encomenda e a atribuição de uma oferta de entrega. O fluxo começa quando o estafeta aceita uma oferta através da *mutation* *acceptDeliveryOffer*. O *DeliveryService* valida a oferta, altera o seu estado de *PENDING* para *ACCEPTED*, associa o estafeta à entrega e expira outras ofertas pendentes para a mesma entrega.

Depois da aceitação, o estado do estafeta é atualizado para *BUSY* através do *CourierService*, e são registados eventos de domínio relacionados com a entrega e com a encomenda. O *OrderService* marca a encomenda como tendo estafeta atribuído, gerando o evento

ORDER_COURIER_ASSIGNED. Este evento é publicado através do *outbox* e propagado em tempo real para o cliente, restaurante e canal de *tracking* da encomenda.

Durante a execução da entrega, o estafeta atualiza o estado através de *mutations* GraphQL. Ao recolher o pedido no restaurante, é chamada *markDeliveryPickedUp*, que muda a entrega de PENDING para PICKED_UP e regista o evento DELIVERY_PICKED_UP. Quando o estafeta inicia o trajeto até ao cliente, a *mutation markDeliveryInTransit* altera a entrega para IN_TRANSIT e o OrderService atualiza a encomenda para OUT_FOR_DELIVERY.

Quando a entrega é concluída, o estafeta chama *markDeliveryDelivered*. O DeliveryService altera a entrega para DELIVERED, regista o evento correspondente, atualiza a encomenda para DELIVERED e coloca novamente o estafeta como AVAILABLE. Os eventos finais são publicados para os canais do cliente, da encomenda e do restaurante, garantindo que todos os intervenientes ficam sincronizados com o estado final da entrega.

3.5 Máquinas de Estado

Para além dos fluxos de interação, foi modelado o ciclo de vida das três entidades centrais através de máquinas de estado. As transições são validadas pelas implementações descritas em 3.2.2.1 Padrão *State*, e cada transição gera obrigatoriamente um evento de auditoria.

3.5.1 Máquina de Estados — Order

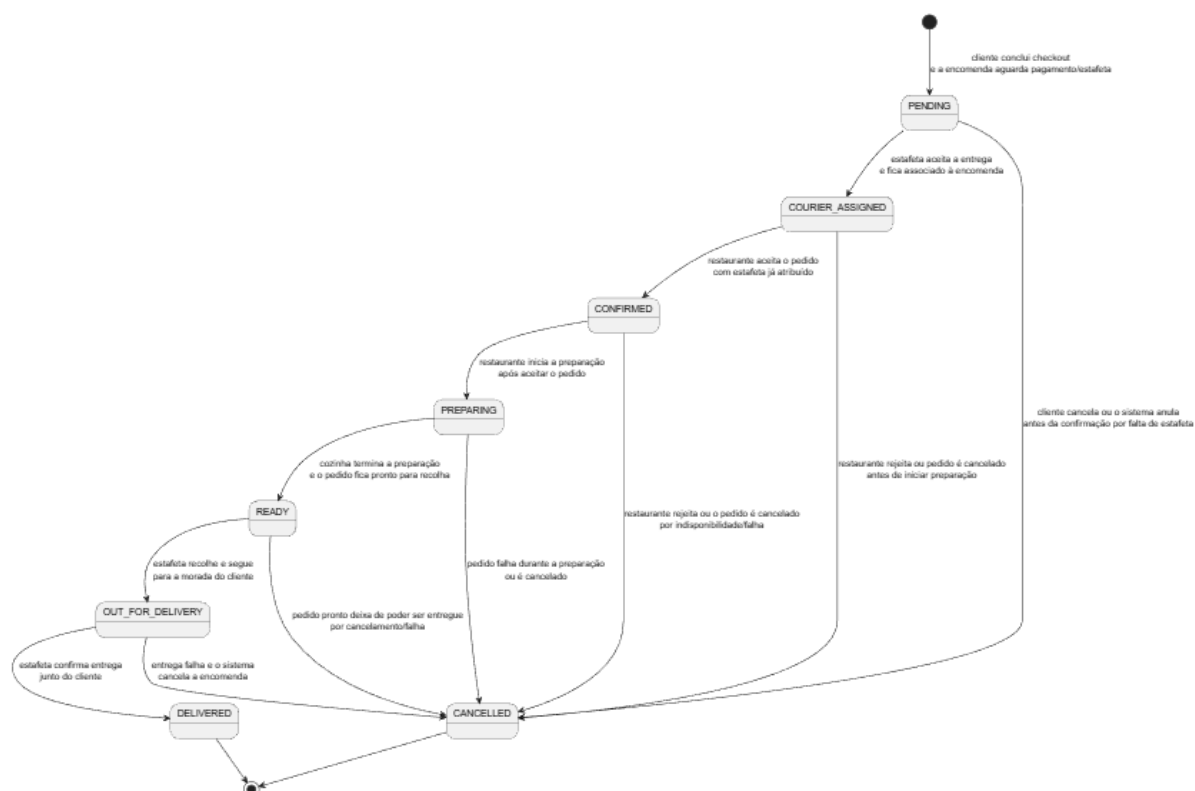


Figura 7 Diagrama de Estados: Order

A encomenda é criada em PENDING após o *checkout* e permanece nesse estado enquanto aguarda a confirmação do pagamento, a criação da entrega e a aceitação de uma oferta por parte de um estafeta. Quando o estafeta aceita a entrega, a encomenda passa para COURIER_ASSIGNED, permitindo ao restaurante aceitar e iniciar a preparação. Depois, a

encomenda evolui para CONFIRMED, PREPARING, READY, OUT_FOR_DELIVERY e finalmente DELIVERED. O estado CANCELLED representa cancelamentos por parte do cliente, rejeição do restaurante, falhas de pagamento ou impossibilidade de atribuir estafeta.

3.5.2 Máquina de Estados — Payment

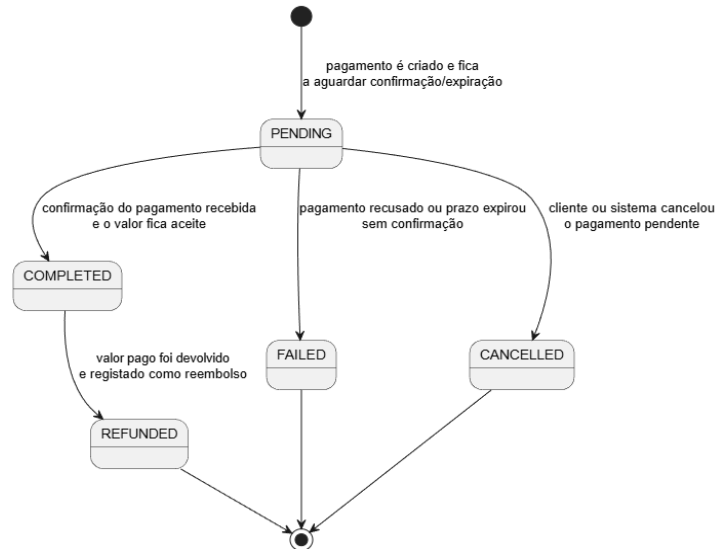


Figura 8 Diagrama de Estados: Payment

O diagrama de estados do pagamento mostra a evolução de um pagamento desde a sua criação até ao seu estado final. Um pagamento começa como PENDING, quando aguarda confirmação. A partir de PENDING, o pagamento pode ser confirmado, falhar por expiração ou erro, ou ser cancelado. Um pagamento já concluído pode ainda passar para REFUNDED, caso a encomenda seja cancelada posteriormente e seja necessário devolver o valor ao cliente.

3.5.3 Máquina de Estados — Delivery

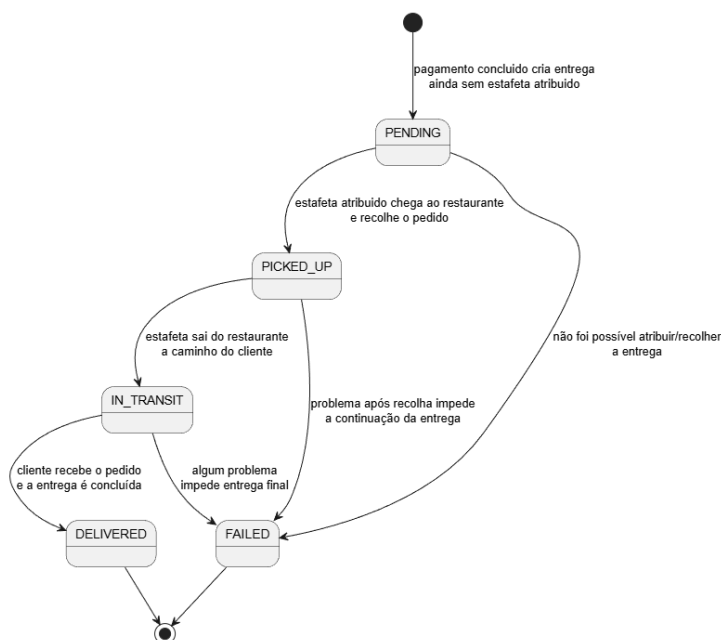


Figura 9 Diagrama de Estados: Delivery

Este diagrama descreve o percurso operacional da entrega associada a uma encomenda. A entrega começa em PENDING depois de ser criada pelo sistema, ainda antes de o pedido ser recolhido. Quando o estafeta chega ao restaurante e recolhe a encomenda, a entrega passa para PICKED_UP. De seguida, quando o estafeta inicia o percurso até ao cliente, passa para IN_TRANSIT. O fluxo termina em DELIVERED, quando o cliente recebe o pedido, ou em FAILED, caso ocorra uma falha durante a atribuição, recolha ou entrega.

Capítulo 4

Implementação da Solução

Neste capítulo é descrita a implementação da plataforma, detalhando as principais decisões técnicas, a organização do sistema e as tecnologias utilizadas. A implementação segue diretamente a arquitetura definida nos capítulos anteriores, baseada numa abordagem cliente-servidor com suporte a comunicação em tempo real.

4.1 Arquitetura Implementada

A arquitetura final do sistema foi implementada com uma organização modular, centrada na separação de responsabilidades entre a API *backend*, os clientes *frontend* e os mecanismos assíncronos de suporte. Esta estrutura permite que os principais fluxos da aplicação, como gestão de encomendas, carrinho, campanhas, pagamentos, entregas e notificações, sejam evoluídos de forma independente e com menor acoplamento entre componentes.

No *backend*, desenvolvido em Laravel, o código encontra-se organizado em camadas lógicas:

- Camada de Domínio — concentra regras de negócio reutilizáveis e independentes sempre que possível, como cálculos de preços e descontos, regras geográficas, enums e máquinas de estado para encomendas, pagamentos e entregas. Esta camada contém lógica que não depende diretamente da interface GraphQL ou dos clientes *frontend*;
- Camada de Aplicação — é composta sobretudo pelos *services*, responsáveis por orquestrar os *use cases* do sistema. Estes *services* coordenam validações, criação e atualização de entidades, aplicação de promoções/cupões, transições de estado, emissão de eventos e chamadas aos repositórios. Em operações críticas, são usados atributos como `#[Transactional]` para garantir consistência transacional;
- Camada de Persistência e Infraestrutura — inclui os modelos Eloquent, repositórios, *migrations*, *jobs*, *listeners*, integração com o sistema de *outbox*, envio de notificações e comunicação em tempo real. A API GraphQL, implementada com Lighthouse, expõe as operações do sistema através de *queries* e *mutations*, delegando a lógica principal para os *services*.

A comunicação entre componentes é feita maioritariamente através de *services*, DTOs e interfaces de repositório registadas no container do Laravel. Esta abordagem reduz a dependência direta entre módulos e torna mais simples testar regras de negócio, substituir implementações e manter os fluxos principais da aplicação.

4.2 Implementação do Backend

O *backend* tem como principal responsabilidade a gestão do estado do sistema, a validação das regras de negócio e a coordenação da comunicação entre os atores. A escolha do Laravel foi fundamentada em 1.2.1 Laravel (Backend) e mantém-se coerente com a estrutura aqui descrita:

- ecossistema maduro (Eloquent, *Migrations*, *Queues*, *Events*, *Notifications*);
- integração nativa com Lighthouse para GraphQL;
- intergração com websockets através de Workerman/GatewayWorker;
- processamento assíncrono via *Queues* e *Jobs*.

4.2.1 Programação Funcional

A Programação Funcional foi aplicada de forma complementar à arquitetura orientada a objetos, sobretudo em componentes de cálculo, validação e transformação de dados, onde era importante

obter comportamento previsível, reutilizável e facilmente testável. A abordagem foi usada para isolar regras de domínio em funções puras, ou seja, funções que recebem dados de entrada e devolvem um resultado sem alterar diretamente o estado da aplicação, a base de dados ou os modelos.

A implementação é visível, por exemplo, nas classes PricingCalculator e GeoMath. A primeira concentra regras de cálculo relacionadas com preços, subtotais, descontos, taxas de entrega e totais finais, a segunda encapsula cálculos geográficos, como a distância entre coordenadas. Estas funções são determinísticas: para os mesmos argumentos, devolvem sempre o mesmo resultado, o que reduz o acoplamento com Laravel ou com a camada de persistência.

Foram usados os seguintes conceitos funcionais:

- funções puras — métodos como `calculateSubtotal`, `calculateTotal`, `discountAmount` e `distanceKm` não dependem de estado externo e apenas calculam valores;
- higher-order functions — métodos como `pipe` e `sumBy` recebem funções como argumento, permitindo compor transformações, seletores e predicados reutilizáveis;
- closures e arrow functions — usadas para definir transformações curtas e predicados, incluindo filtragem de itens, mapeamento de DTOs/snapshots e cálculo parametrizado de descontos;
- composição de funções — operações como multiplicar por quantidade, aplicar desconto e arredondar valores são encadeadas em vez de serem escritas como blocos imperativos extensos;
- transformação declarativa de coleções — utilização de métodos como `map`, `filter`, `flatMap`, `pluck`, `sum`, `values` e `all` nas Collections do Laravel, tornando mais clara a intenção do código;

A utilização deste paradigma trouxe resultados benéficos ao desenvolvimento do sistema, tais como:

- maior previsibilidade nos cálculos críticos do domínio;
- menor duplicação de lógica de cálculo entre *services*;
- código mais declarativo e legível em operações sobre listas e coleções;
- facilidade de manutenção, uma vez que alterações nas regras de preço ou distância ficam concentradas em funções pequenas e reutilizáveis.

4.2.2 Programação Orientada a Aspectos

A Programação Orientada a Aspectos foi aplicada para separar preocupações transversais da lógica principal do sistema, nomeadamente a gestão de transações, o *logging* de execução e o registo de exceções. A implementação foi feita através da biblioteca `ray/aop`, integrada no `RayAopServiceProvider`, que regista `Ray\Aop\Aspect` como *singleton* e cria *proxies* para os *services*, *listeners* e *handlers* configurados.

A abordagem assenta em intercetores associados a atributos nativos do PHP 8, como `#[Transactional]`, `#[Logged]` e `#[ErrorLogged]`. Desta forma, é possível adicionar comportamento transversal a métodos específicos sem misturar esse código com as regras de negócio.

Foram definidos os seguintes aspetos:

- Gestão de transações — métodos críticos anotados com `#[Transactional]` são automaticamente executados dentro de uma transação da base de dados através de `DB::connection()->transaction(...)`. Em caso de sucesso, a transação é confirmada; se ocorrer uma exceção, o Laravel garante o *rollback*. Isto evita repetir código de controlo transacional nos *services* e mantém operações como *checkout*, atualização de estados e aceitação de entregas mais limpas.
- *Logging* de execução — o `LoggingInterceptor` regista o início e o fim da execução dos métodos *proxied*, incluindo a classe, o método e a duração em milissegundos. Quando o método está anotado com `#[Logged]`, é feito também um registo adicional no nível de *log* indicado pelo atributo.
- Registo transversal de exceções — o `ErrorHandlingInterceptor` captura exceções lançadas pelos métodos *proxied*, regista informação como classe, método, tipo de exceção e mensagem, e volta a lançar a exceção para que o fluxo normal de erro da aplicação seja preservado. Em métodos anotados com `#[ErrorLogged]`, o *log* pode ser configurado.

A utilização deste paradigma trouxe resultados benéficos ao desenvolvimento do sistema, tais como:

- maior separação de responsabilidades (regras de negócio vs. infraestrutura);
- código mais limpo e legível, focado no domínio;
- consistência: todos os métodos críticos seguem a mesma política transacional;
- facilidade de evolução: adicionar um novo aspeto (e.g., invalidação de *cache*, métricas) requer apenas um novo *interceptor*, sem tocar nos *services* existentes.

4.2.3 Programação Orientada a Eventos

A Programação Orientada a Eventos foi aplicada como mecanismo central de comunicação entre os componentes do sistema, permitindo que ações de domínio desencadeiem reações sem que os módulos emissores conheçam diretamente os recetores.

Um dos exemplos mais relevantes encontra-se no modelo *Order*. Este modelo define um mecanismo próprio de reação a mudanças de estado através do método estático `onStatusChanged`. Durante a construção do modelo, é registado o `OrderStatusChangedHandler` como handler de alterações de estado. Sempre que uma encomenda muda de estado, a transição não atualiza simplesmente o campo `status` diretamente; em vez disso, a *state machine* chama `Order::transitionTo(...)`, que guarda o estado anterior, atualiza o novo estado e agenda a execução dos *handlers* através de `DB::afterCommit`.

Esta decisão é importante porque garante que os eventos relacionados com a encomenda só são emitidos depois que a transação da base de dados seja confirmada. Assim, evita-se que outros componentes reajam a uma alteração que poderia ser revertida por *rollback*. Após o *commit*, o `OrderStatusChangedHandler` recebe a encomenda, o estado anterior e o novo estado, constrói o *payload* do evento e regista um `OrderEvent` no histórico da encomenda. De seguida, cria também um evento na *outbox* através do `OutboxService`.

O tipo de evento publicado é derivado do novo estado da encomenda através do método `OrderStatus::toEventType`. Por exemplo, uma transição para `COURIER_ASSIGNED` origina o evento `ORDER_COURIER_ASSIGNED`, uma transição para `PREPARING` origina `ORDER_PREPARING`, e uma transição para `DELIVERED` origina `ORDER_DELIVERED`. Desta forma, a lógica de transição de estados fica centralizada e consistente com os eventos emitidos para o resto do sistema.

A *outbox* funciona como uma fila persistente de eventos de domínio. Em vez de publicar imediatamente o evento para WebSocket, notificações ou outros consumidores, o sistema grava primeiro um registo numa tabela própria, com informação como o tipo de agregado, o identificador do agregado, o tipo de evento, o *payload* e o estado de publicação. Quando o evento é criado, começa no estado PENDING. Como esta gravação ocorre no mesmo fluxo transacional da alteração de domínio, garante-se que a mudança de estado da encomenda e o evento correspondente ficam consistentes entre si.

Depois de registado na *outbox*, o evento é publicado de forma assíncrona pelo `PublishOutboxEventJob`. Este *job* é despachado apenas após o *commit* da transação e chama o `OutboxService` para processar o registo. Durante a publicação, o evento passa para PROCESSING e é emitido o evento genérico `OutboxEventPublished`. Este evento é processado pelo *listener* `DispatchLaravelEventFromOutbox`, que mapeia o tipo do evento para o *handler* adequado. Esse *handler* reemite eventos tipados do Laravel, como `OrderStatusUpdated`, `CourierAssigned`, `DeliveryOffered` ou `CourierPositionUpdated`.

Estes eventos tipados podem então ser consumidos por *listeners* independentes, como o `DispatchSocketMessage`, responsável por enviar atualizações em tempo real para os clientes ligados por WebSocket. Assim, uma transição de estado numa encomenda pode atualizar automaticamente o histórico, alimentar a *outbox*, publicar eventos, atualizar interfaces em tempo real e gerar notificações, sem que o *service* que executou a transição precise de conhecer esses consumidores.

Este fluxo permite que uma simples mudança de estado numa encomenda desencadeie várias consequências sem acoplamento direto entre módulos: persistência do histórico da encomenda, criação de eventos de outbox, publicação assíncrona, notificações em tempo real e eventuais notificações ao utilizador. O *service* que muda o estado da encomenda apenas executa a transição de domínio, o restante comportamento é tratado por *handlers*, *jobs* e *listeners* especializados.

4.3 API GraphQL

A API GraphQL foi implementada como principal *interface* de comunicação estruturada entre os clientes e o servidor, recorrendo ao *package* Lighthouse, que permite definir o *schema* em ficheiros `.graphql` decorados com *directives* para mapear *queries* e *mutations* aos *resolvers* (ou diretamente aos métodos de *service*)

Segue-se a modularização do *schema*:

- `schema.graphql` — *root* e *includes*;
- `common.graphql` — tipos partilhados (`User`, `Address`, *scalars*);
- `users.graphql`, `restaurants.graphql`, `menus.graphql`, `carts.graphql`, `orders.graphql`, `payments.graphql`, `deliveries.graphql`, `tracking.graphql`, `notifications.graphql`, `reviews.graphql`, `campaigns.graphql`, `chat.graphql`, ou seja, *schemas* por domínio.

Os *resolvers* delegam a lógica de negócio para a camada de Aplicação, evitando a introdução de lógica complexa na camada de API e promovendo uma clara separação de responsabilidades.

A utilização de GraphQL reduz o *over-fetching* porque cada cliente pode pedir apenas os campos de que necessita para cada ecrã, em vez de receber uma resposta fixa e demasiado extensa. Isto é útil porque a aplicação tem interfaces com necessidades diferentes: o cliente mobile precisa de dados de restaurantes, carrinho, checkout e *tracking*, o estafeta precisa de ofertas e estados de entrega, e o restaurante precisa de encomendas, itens e preparação.

4.4 Comunicação em Tempo Real com Workerman/GatewayWorker

A comunicação em tempo real foi implementada com Workerman/GatewayWorker, recorrendo às dependências workerman/gateway-worker e workerman/gatewayclient. A integração encontra-se organizada em dois fluxos principais: mensagens recebidas dos clientes pelo servidor WebSocket e eventos emitidos pelo backend para os clientes ligados.

No fluxo cliente-servidor, a classe GatewayWorkerEvents funciona como ponto de entrada do GatewayWorker, tratando eventos do ciclo de vida da ligação como onWorkerStart, onConnect, onMessage e onClose. Quando uma mensagem chega através de onMessage, esta é encaminhada para o ClientEventDispatcher, que interpreta o tipo da mensagem e a delega para o *handler* adequado. Entre os tipos suportados estão hello, subscribe, unsubscribe, chat.message.send, courier.status.set e courier.position.set. O evento hello associa a ligação a um utilizador através de Gateway::bindUid e coloca o cliente em grupos base, como user.{userId}.notifications ou courier.{courierId}.jobs.

No fluxo servidor-cliente, os eventos do domínio são recebidos pelo listenerDispatchSocketMessage, que delega para o SocketMessageDispatcher. Este percorre *handlers* especializados, como OrderStatusUpdatedSocketHandler, CourierAssignedSocketHandler, DeliveryOfferedSocketHandler, CourierPositionUpdatedSocketHandler, ChatMessageSentSocketHandler e UserNotificationCreatedSocketHandler. Cada *handler* verifica se suporta o evento recebido e, em caso afirmativo, envia a mensagem para o grupo correto através do GatewayClientSocketPusher, que utiliza o GatewayClient para comunicar com o GatewayWorker.

A comunicação é organizada em canais segmentados por contexto, garantindo que cada cliente recebe apenas os eventos relevantes para si. Por exemplo, atualizações de encomenda são enviadas para canais como order.{orderId}.tracking, customer.{customerId}.orders e restaurant.{restaurantId}.orders, ofertas de entrega são enviadas para courier.{courierId}.jobs, mensagens de chat para chat.{chatId}, e notificações para user.{userId}.notifications. Os clientes podem ainda subscrever ou remover subscrições dinamicamente através das mensagens *subscribe* e *unsubscribe*.

Grande parte dos eventos enviados por WebSocket tem origem no mecanismo de *outbox*. Quando ocorre uma alteração relevante no domínio, como uma mudança de estado da encomenda, o evento é persistido e publicado de forma assíncrona. Depois, é convertido em eventos Laravel tipados, como OrderStatusUpdated, CourierAssigned ou DeliveryOffered, que acabam por ser encaminhados para os respetivos *handlers* WebSocket.

Esta arquitetura evita *polling* constante por parte dos clientes, reduz carga desnecessária sobre a API e a base de dados, e permite que alterações importantes sejam refletidas em tempo real nas interfaces do cliente, restaurante e estafeta.

4.5 Implementação dos Frontends

O sistema inclui dois *frontends*, com responsabilidades segmentadas conforme os atores que servem: o *frontend web* dedicado ao restaurante e o *frontend mobile* que cobre cliente e estafeta num único binário.

4.5.1 Frontend Web (React/Vite) – Restaurante

O *frontend web* foi desenvolvido em React com o *bundler* Vite, centrado na experiência de gestão do restaurante. A estrutura segue uma abordagem *feature-first*:

- `components/` – componentes UI partilhados;
- `features/` – módulos funcionais organizados por contexto;
- `screens/` – ecrãs raiz;
- `services/` – clientes GraphQL, serviços de WebSocket, serviços auxiliares;
- `utils/` – utilitários comuns.

Ecrãs principais:

- **RestaurantOrdersQueueScreen** – ecrã principal do restaurante, que funciona como dashboard operacional. Apresenta a fila de pedidos ativos em tempo real, subscrevendo o canal `restaurant.{id}.orders`. Mostra o estado das encomendas, hora de criação, valor e métricas resumidas, permitindo acompanhar rapidamente o fluxo de trabalho do restaurante;
- **RestaurantOrderDetailScreen** – detalhe de uma encomenda específica, com informação do cliente, estado atual, itens, opções, histórico e dados de entrega. Disponibiliza ações operacionais como aceitar pedido, iniciar preparação e marcar a encomenda como pronta;
- **RestaurantVirtualKitchenScreen** – vista orientada à cozinha, focada nos itens individuais das encomendas em preparação. Permite atualizar o estado de cada item e facilita a coordenação da preparação. Quando todos os itens de uma encomenda ficam prontos, o *backend* pode atualizar a encomenda para READY;
- **RestaurantOrdersHistoryScreen** – histórico de encomendas do restaurante, permitindo consultar pedidos anteriores ou filtrar encomendas por estados;
- **RestaurantMenuCatalogScreen** – gestão do menu do restaurante, incluindo produtos, categorias, opções e disponibilidade dos itens;
- **RestaurantChainCatalogScreen** – gestão do catálogo ao nível da cadeia, quando aplicável, permitindo organizar produtos partilhados entre restaurantes da mesma cadeia;
- **RestaurantCampaignsScreen** – gestão de campanhas promocionais, permitindo criar, listar, ativar ou desativar campanhas e associá-las a produtos ou condições específicas;
- **RestaurantChatScreen** – ecrã de chat associado às encomendas, com subscrição ao canal `chat.{id}`, permitindo comunicação em tempo real com outros participantes;
- **RestaurantNotificationsScreen** – listagem de notificações recebidas pelo restaurante/utilizador, integrada com o canal `user.{id}.notifications`;
- **RestaurantReviewsScreen** – consulta das avaliações recebidas pelo restaurante, permitindo acompanhar feedback dos clientes;
- **RestaurantProfileScreen** – gestão de dados do perfil do restaurante, incluindo informação operacional e dados de localização.

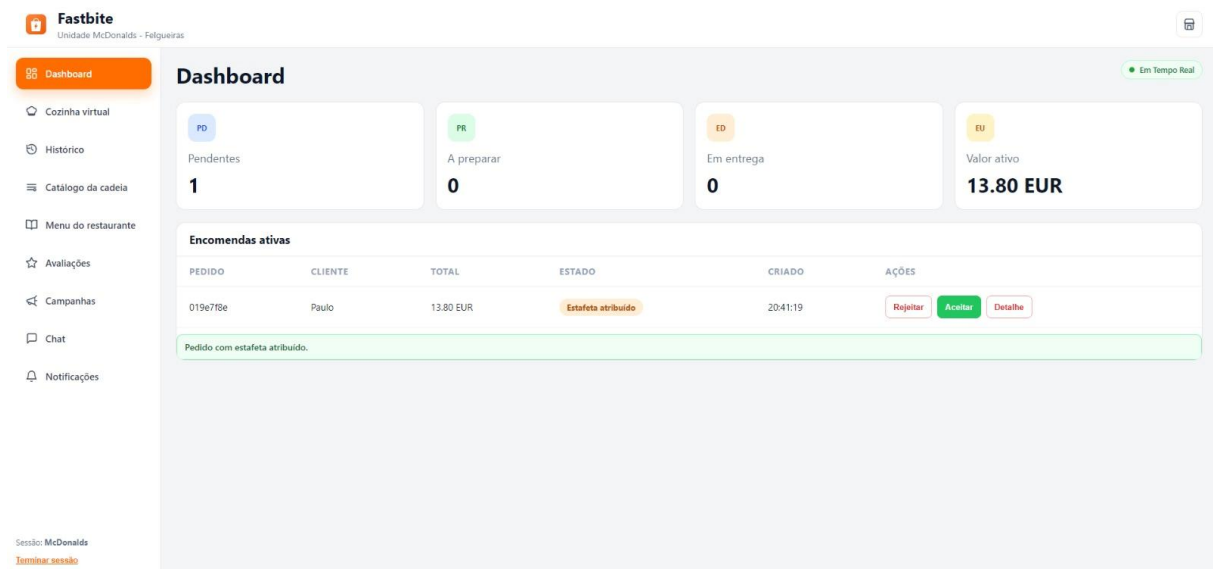


Figura 10 Frontend Dashboard Restaurante

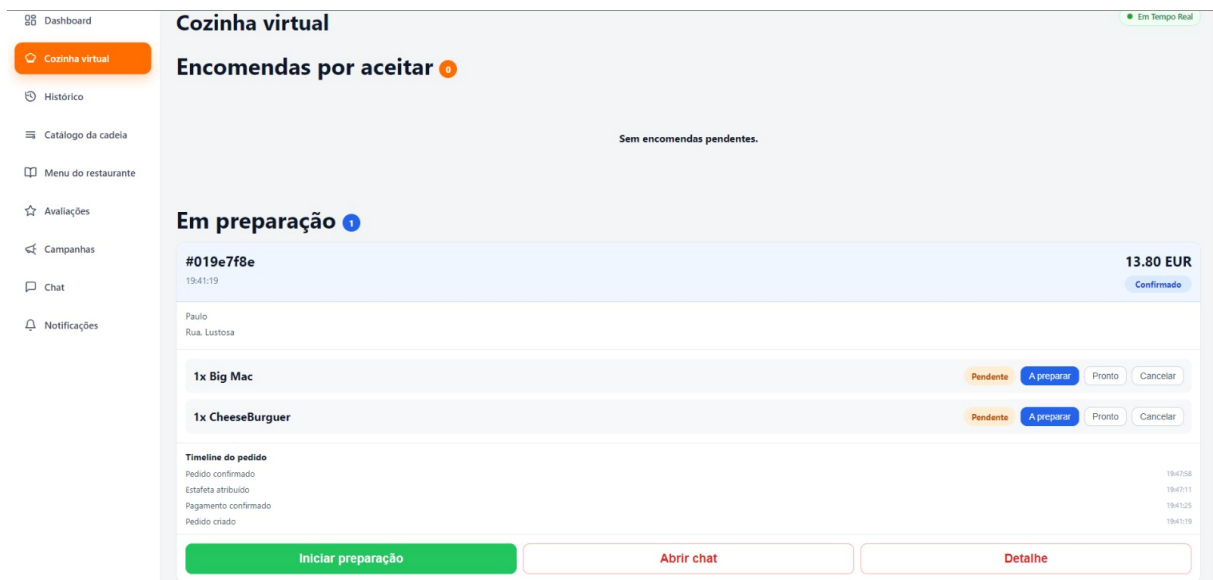


Figura 11 Frontend Cozinha Virtual

4.5.2 Frontend Mobile (React Native/Expo) – Cliente + Estafeta

A aplicação mobile foi desenvolvida em React Native com Expo, cobrindo num único binário os perfis de cliente e estafeta. A estrutura organiza-se principalmente em:

- components/ – componentes UI partilhados, incluindo mapas, badges, botões e elementos visuais comuns;
- screens/ – ecrãs principais da aplicação, incluindo o fluxo do cliente, o fluxo do estafeta e o ecrã de login;
- screens/customer/ – ecrãs e lógica específica do cliente, incluindo rotas internas em CustomerRoutes;
- screens/courier/ – estilos e utilitários associados ao fluxo do estafeta;
- services/ – clientes GraphQL, comunicação realtime, localização, notificações *push* e serviços de domínio como carrinho, encomendas, pagamentos, *tracking* e estafeta.

Fluxo do Cliente (screens/customer/):

- HomeScreen — *home* com restaurantes disponíveis, destaques e categorias;
- MenuScreen — menu do restaurante selecionado, com produtos, categorias e opções;
- CartScreen — carrinho com itens, opções, totais e botão de *checkout*;
- OrdersHistoryScreen — histórico de encomendas, com filtros por estado;
- TrackingScreen — acompanhamento da entrega em tempo real: mapa com restaurante, endereço de entrega e posição animada do estafeta, ETA estimado, estado atual da encomenda, botão para *chat*;
- ProfileScreen — perfil do utilizador e gestão de moradas.

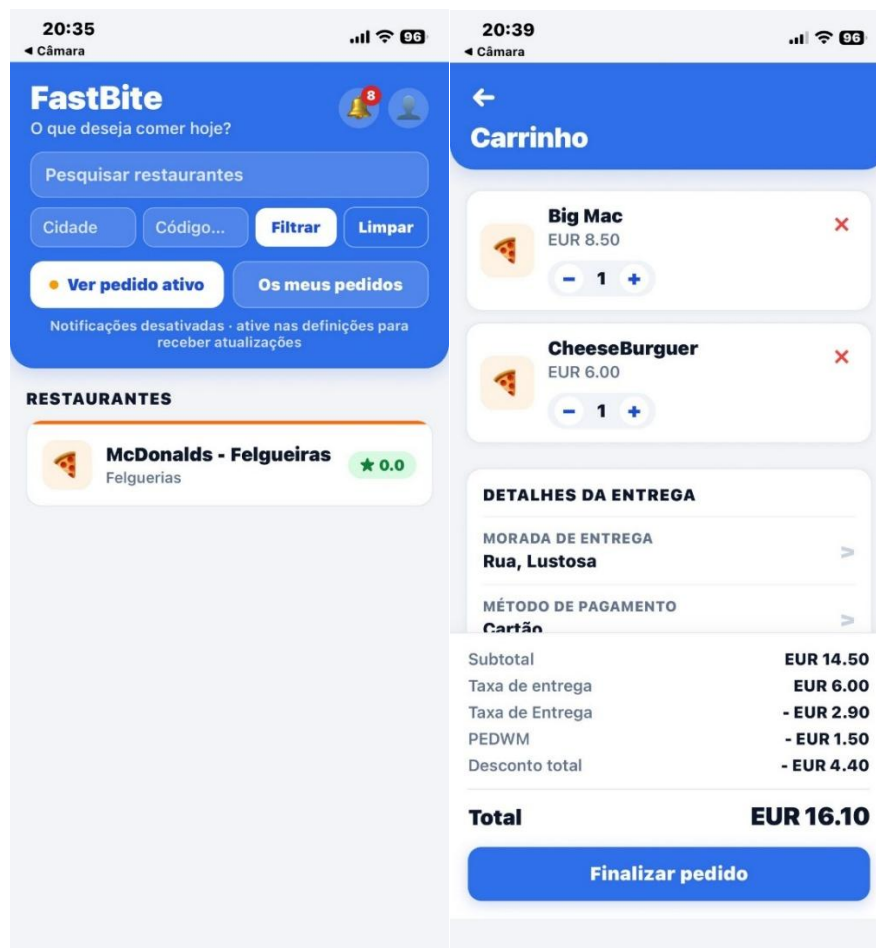


Figura 12 Frontend Cliente Home e Carrinho

Fluxo do Estafeta (CourierAppScreen e dependências):

- Gestão de disponibilidade — o estafeta pode alternar o seu estado operacional, ficando disponível para receber novas propostas de entrega ou indisponível para o sistema;
- Receção de ofertas de entrega — quando é emitido um evento JOB_OFFERED, a aplicação apresenta a proposta ao estafeta com informação da entrega, tempo limite de resposta e ações para aceitar ou rejeitar a oferta;
- Execução da entrega — após aceitar uma oferta, o estafeta acompanha o fluxo operacional da entrega, desde a deslocação até ao restaurante, recolha do pedido, início do percurso até ao cliente e confirmação da entrega concluída;
- Atualização de estados — o estafeta interage com a aplicação para marcar momentos importantes, como recolha do pedido, início do trajeto em direção ao cliente e entrega final;
- Transmissão de localização — durante a entrega ativa, a aplicação envia periodicamente a localização do estafeta através de expo-location.

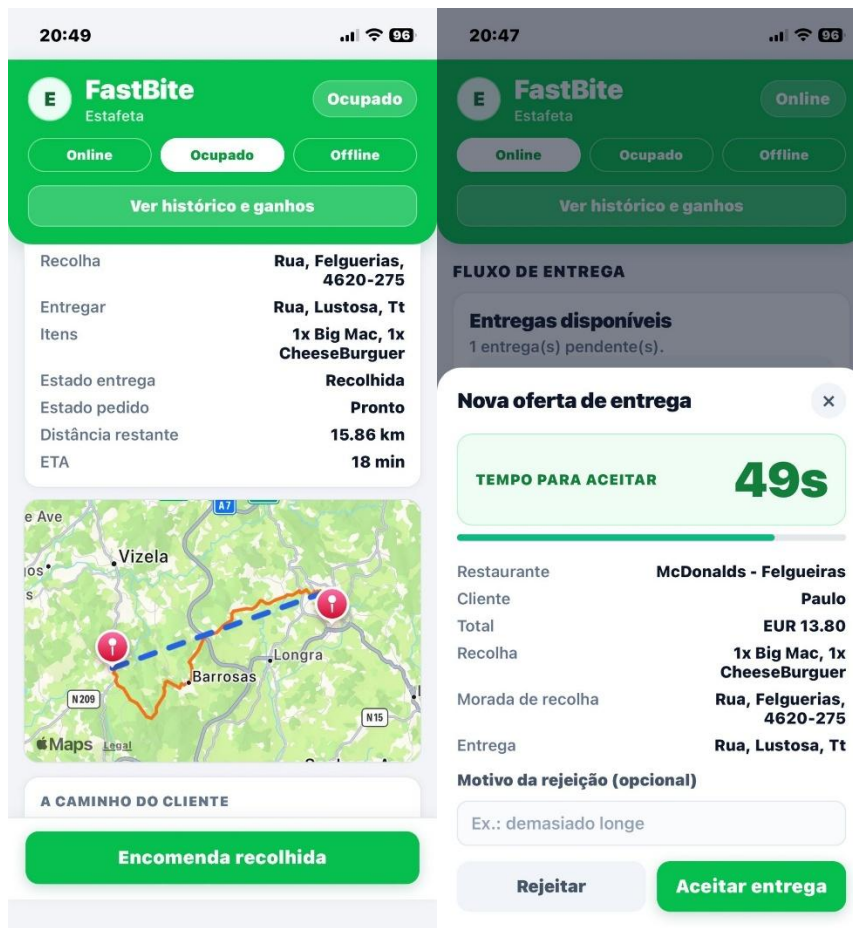


Figura 13 Frontend Estafeta A Recolher Encomenda e Aceitar Encomenda

4.6 Organização do Projeto e Gestão do Trabalho

O desenvolvimento do projeto foi realizado de forma iterativa, com base numa abordagem inspirada em metodologias ágeis principalmente com a atribuição equilibrada de *story points*.

A gestão do projeto foi suportada por ferramentas de acompanhamento e controlo de tarefas, permitindo manter uma visão clara do progresso do trabalho e das responsabilidades de cada elemento do grupo.

O planeamento e acompanhamento do trabalho foram realizados com recurso ao **Jira**, onde foram definidas tarefas para cada um dos elementos, de forma a balanceada quanto à quantidade de *story points*. Para além disso estas tarefas foram geridas através dos seus estados de progresso (a fazer, em desenvolvimento, concluído).

Esta abordagem permitiu uma distribuição equilibrada do trabalho e facilitou o acompanhamento da evolução do projeto.

No final do projeto, cada membro do grupo completou a seguinte quantidade de *story points*:

- Alexandre Freitas – 22 *story points*;
- Duarte Sousa – 28 *story points*;
- Paulo Coelho – 26 *story points*.

O código-fonte do projeto foi versionado utilizando **Git**, com alojamento num repositório [Github](#).

Capítulo 5

Discussão dos Resultados e Conclusão

5.1 Apresentação e discussão de resultados

O desenvolvimento da plataforma FastBite permitiu concretizar os principais objetivos definidos para o projeto, resultando numa solução funcional orientada à gestão de encomendas e entregas de refeições. A aplicação desenvolvida permite a interação entre três tipos de utilizadores distintos (cliente, restaurante e estafeta) assegurando que cada um dispõe de funcionalidades adequadas ao seu papel no ciclo de vida de uma encomenda.

A solução implementada integra um *backend* desenvolvido em Laravel, responsável pela lógica de negócio, persistência de dados, gestão de estados e coordenação entre os vários intervenientes. Para a camada de apresentação, foram desenvolvidos dois clientes complementares: uma aplicação web em React/Vite, direcionada para a gestão operacional dos restaurantes, e uma aplicação mobile em React Native/Expo, destinada aos fluxos do cliente e do estafeta.

Um dos resultados mais relevantes foi a adoção de uma arquitetura híbrida baseada em GraphQL e WebSockets. O GraphQL foi utilizado para operações estruturadas, como consultas, criação de encomendas, gestão de carrinhos, atualização de entidades e consulta de histórico. Esta abordagem permitiu que cada cliente solicitasse apenas os dados necessários para cada contexto, reduzindo problemas de over-fetching e under-fetching.

Por outro lado, os WebSockets foram utilizados para assegurar a comunicação em tempo real entre os diferentes atores do sistema. Esta componente revelou-se essencial para a atualização imediata do estado das encomendas, propagação de notificações, acompanhamento da localização do estafeta e sincronização entre cliente, restaurante e estafeta. Desta forma, a arquitetura adotada demonstrou ser adequada ao contexto de uma plataforma de delivery, onde a rapidez e consistência da informação são fatores críticos.

A nível funcional, foi possível representar o ciclo de vida principal de uma encomenda, desde a seleção de produtos e checkout até à preparação pelo restaurante, atribuição de estafeta, recolha, transporte e entrega. Também foram modeladas entidades relevantes do domínio, como utilizadores, restaurantes, produtos, carrinhos, encomendas, pagamentos, entregas, notificações, promoções, chat e avaliações. Esta modelação permitiu estruturar o sistema de forma coerente e preparar a aplicação para futuras extensões.

5.2 Trabalho Futuro

Como trabalho futuro, seria relevante aprofundar os mecanismos de segurança da plataforma, tornando a autenticação e autorização mais robustas e adequadas a um ambiente real. A autorização por perfil e por canal WebSocket seria uma melhoria importante para garantir que cada utilizador apenas tem acesso aos dados e eventos que lhe dizem respeito.

Outro ponto de evolução seria a integração com *gateways* reais de pagamento, permitindo substituir a simulação atual por transações reais. Esta integração deveria manter o ciclo de estados já modelado, aproveitando a estrutura existente do sistema.

Também seria importante melhorar a escalabilidade da comunicação em tempo real. Para isso, poderiam ser introduzidos mecanismos de balanceamento, gestão distribuída de canais WebSocket e filas de mensagens mais robustas.

No domínio da logística, a atribuição de estafetas poderia ser evoluída para um algoritmo mais inteligente, tendo em conta distância, disponibilidade, tempo estimado de chegada, histórico de entregas e carga atual de trabalho.

Por fim, poderiam ser adicionadas funcionalidades complementares como dashboards analíticos mais completos para restaurantes, sistemas de recomendação de produtos, previsão de procura, gestão avançada de campanhas e métricas de desempenho da operação.

5.3 Conclusão

Conclui-se que o desenvolvimento da FastBite permitiu construir uma solução funcional e coerente para a gestão de encomendas e entregas de refeições, aplicando de forma integrada tecnologias modernas e paradigmas emergentes de programação.

A combinação entre Laravel, PostgreSQL, GraphQL, WebSockets, React/Vite e React Native/Expo permitiu implementar uma arquitetura capaz de responder às necessidades de um sistema distribuído e multiutilizador. A separação entre *backend*, *frontend web* e aplicação *mobile* contribuiu para uma organização clara da solução e para a adaptação da interface às necessidades específicas de cada ator.

A aplicação prática dos paradigmas de Programação Orientada a Eventos, Programação Funcional e Programação Orientada a Aspectos demonstrou-se relevante para lidar com os desafios do domínio. A orientação a eventos permitiu reagir às alterações do ciclo de vida das encomendas e entregas, a programação funcional contribuiu para regras de cálculo mais previsíveis, e a orientação a aspectos permitiu separar preocupações transversais da lógica principal do sistema.

Além disso, a utilização de design patterns como State, Observer e Factory permitiu estruturar melhor a lógica da aplicação, reduzir acoplamento e facilitar a evolução futura. Estes padrões revelaram-se particularmente importantes na gestão de estados, propagação de eventos e organização das responsabilidades internas do sistema.

Apesar de existirem limitações, nomeadamente ao nível da segurança, pagamentos reais e escalabilidade da camada WebSocket, o projeto cumpriu os objetivos definidos e demonstrou a viabilidade da abordagem adotada. A FastBite evidencia, assim, a importância da modelação explícita do domínio, da separação de responsabilidades e da escolha adequada de paradigmas e padrões no desenvolvimento de sistemas reativos e extensíveis.

Anexos

Anexo A – Frontend Web

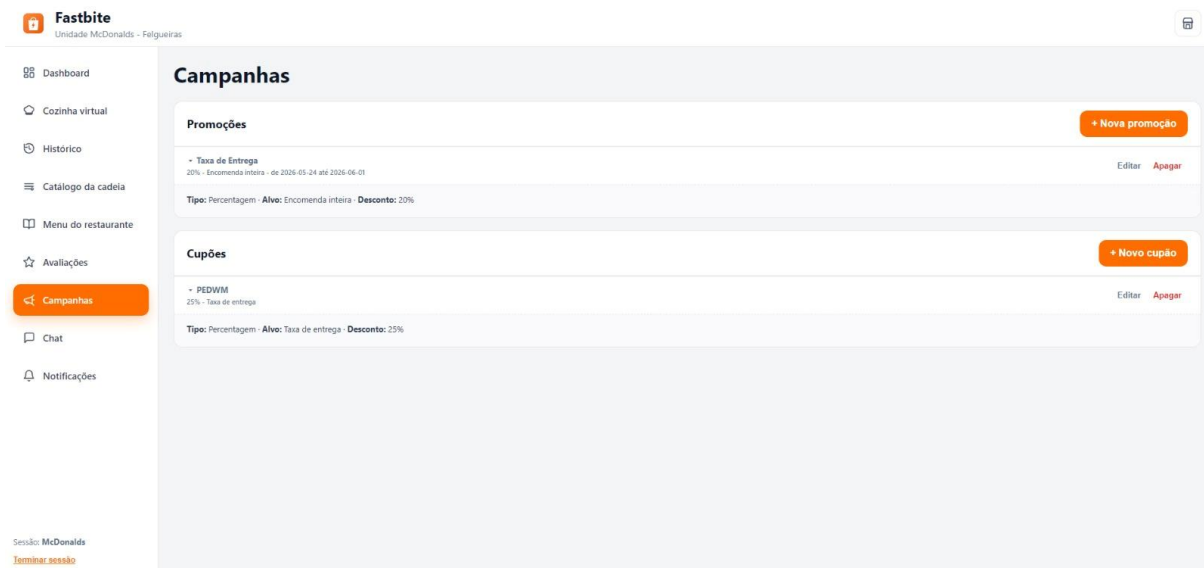


Figura 14 Frontend Restaurante campanhas

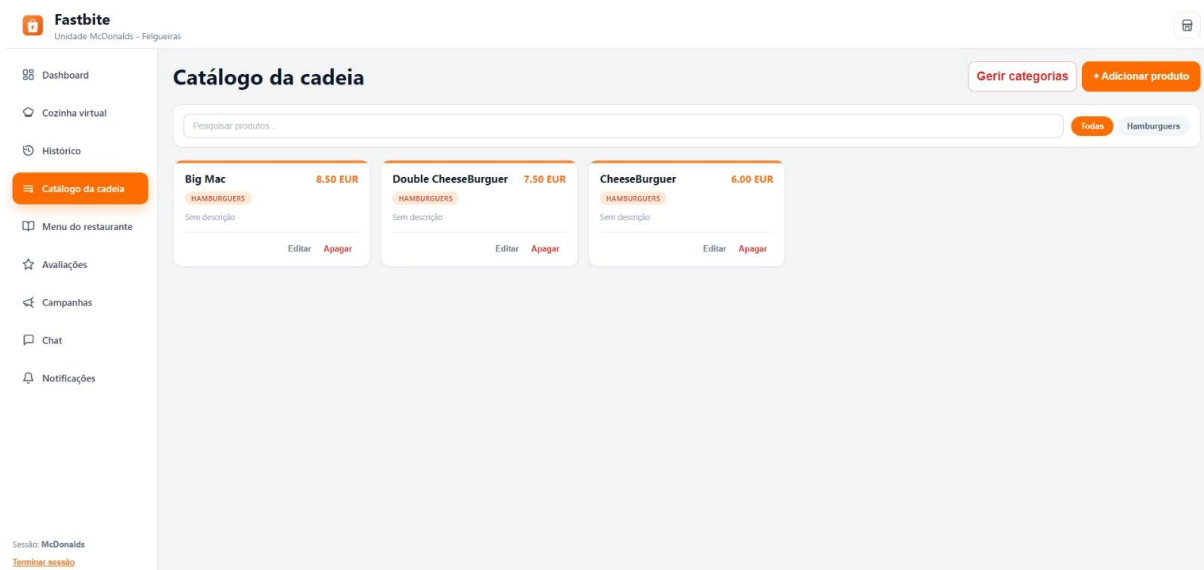


Figura 15 Frontend Catálogo Produtos Restaurante

Anexo B – Frontend Mobile Cliente

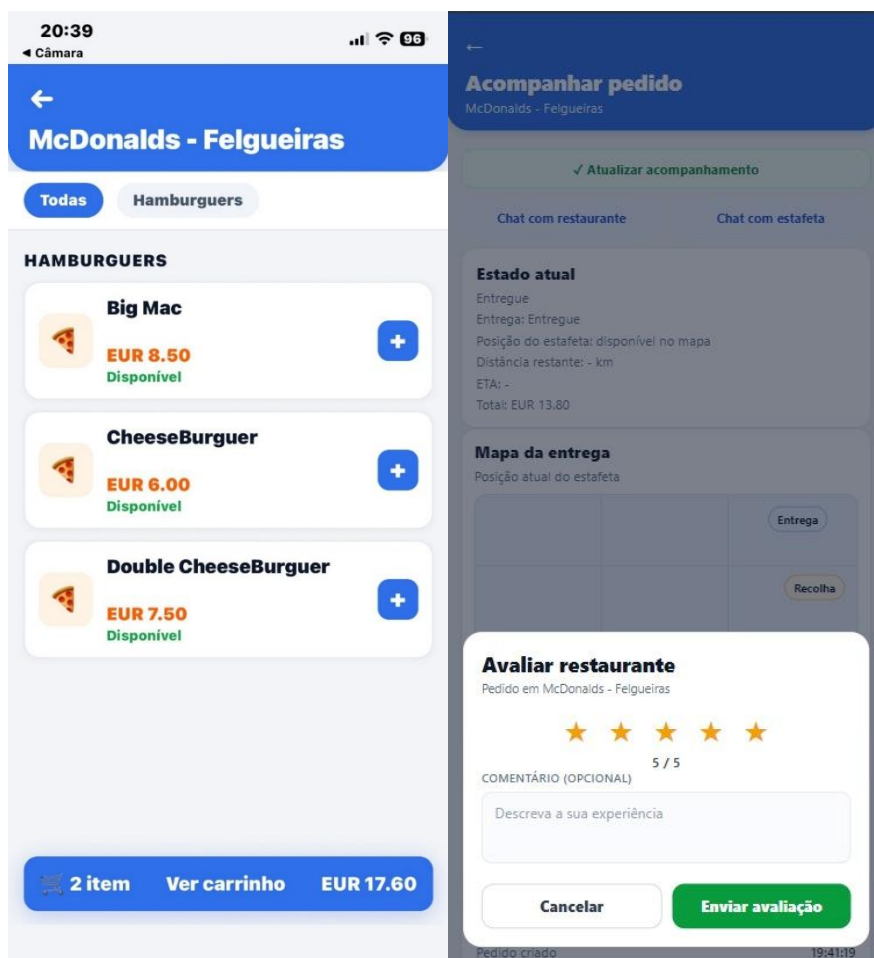


Figura 16 Frontend Cliente Carrinho e Avaliação

Anexo C – Frontend Mobile Estafeta

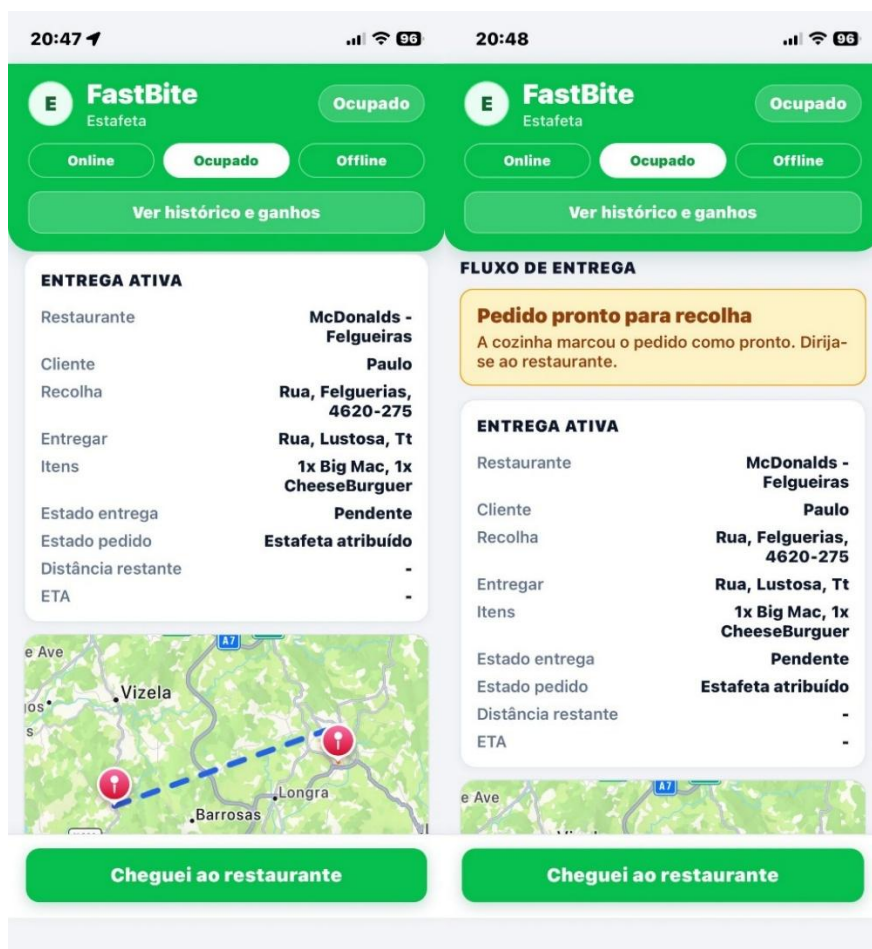


Figura 17 Frontend Estafeta A Chegar e Pedido Pronto a Recolher

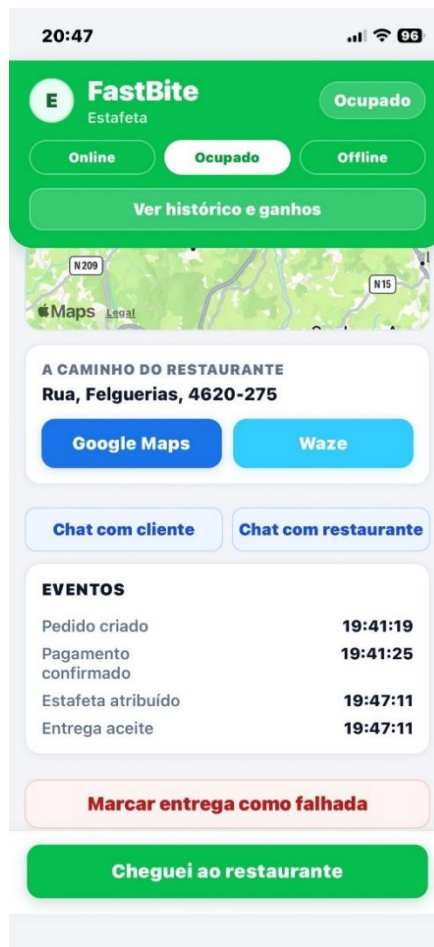


Figura 18 Frontend Estafeta